WannaCry Malware Analysis

A full malware sample analysis which was later discovered to be the well-known ransomware "WannaCry"

Oliver Wright

CMP320: Advanced Ethical Hacking

2024/25

Note that Information contained in this document is for educational purposes.

Abstract

Malware attacks are an ongoing issue in the current age as technology evolves, it creates more and more potential hosts and vulnerabilities for malware to exploit. That is why it is so important to gain an understanding of these malicious processes to fight back against them in the ongoing battle of cybersecurity. Malware analysis is the best way to do this as it can be used to break down attacks that are currently happening and help develop a fix for the attack in real time. WannaCry is a devastating ransomware that has been estimated to cost the NHS (National Health Service) alone over £92 million (Hughes, 2018).

This report details the methodology behind conducting a comprehensive malware analysis, focusing on a variant of the WannaCry ransomware. By using a combination of static analysis, dynamic analysis and code analysis, the aim was to uncover the inner workings of the WannaCry sample variant. The analysis also investigates the attacker's financial gain from the attack and provides insight into how they securely collected this money through cryptocurrency without being detected. By performing this analysis, this report can help not only gain an advanced understanding of this specific malware sample but also construct a potential methodology to help with analysis of future malware threats.

It was found that WannaCry made use of many methods of trying to hide its intentions before execution, but did not try too hard to remain persistent on the victim's machine after running once. This is because after the malware has run once and encrypted the victim's files, it had no further tasks to perform other than receiving payment, which is up to the user to do. Another interesting find was that this sample of WannaCry did not seem to contain the same level of networking capabilities as its relatives. Although it is noted that this sample may have behaved differently if a better testing environment had been set up for the network analysis. Overall, this project was successful in identifying and documenting the malware sample. It gave strong in-depth insight into how the malware works with evidence supporting all claims. It identifies strengths and weaknesses in the malware's code/ functionality as well as providing information into how malware can be fought, and even overall prevent. Lastly, the project is comprehensively documented in this report.

1 CONTENTS

2 INTRODUCTION

2.1 BACKGROUND

Malware is a piece of software developed to cause harm to a computer system, device or network. It can come in various forms such as spyware, viruses, trojans, worms or ransomware, each having their own purpose to maliciously use a victim's device for personal gain. While all of these are a serious issue, ransomware is by far the most common type of malware. Ransomware is malicious software that often restricts access to a device or the files on the device until a sum of money (ransom) is paid. Some ransomware does this by encrypting all the files on a file system and then decrypting them after the ransom has been paid. More specifically, this is how the well-known ransomware "WannaCry" restricted its victims to encourage payment.

The reason WannaCry is very well known is because it is one of the largest ransomware attacks ever recorded. This malicious piece of software emerged in May of 2017 and caused serious damage to companies and individuals alike. WannaCry had a large impact on the UK NHS (National Health Service) as well as the Spanish mobile company Telefónica. "The WannaCry ransomware attack had a substantial financial impact worldwide. It is estimated this cybercrime caused $4 billion in losses across the globe", (Kaspersky, n.d.). The main reason for WannaCry's success is its ability to worm between different devices on the network, allowing it to rapidly spread after infecting just one machine. It did this by exploiting a zero-day vulnerability, "EternalBlue", in devices that use an older version of Windows SMB (Server Message Block) protocol. This vulnerability was originally developed by the NSA (National Security Agency) who decided not to report it to Microsoft for several years until it was breached by a known hacker group "Shadow Brokers". This group then publicly released the exploit code which was used to develop the WannaCry ransomware as well as other malware.

This report documents a full malware analysis carried out on a sample of the WannaCry malware, specifically the variant identified as "Wana Decrypt0r 2.0". By performing this malware analysis, this project intends to uncover details on how the malware functions, specifically focusing on the encryption process and how it compromises the victim's data. This can be achieved by examining the malware sample with both static and dynamic techniques, allowing a further understanding of the malware's functionality. Furthermore, the analysis also involves inspecting the malware's source code at a machine level to view the exact functions and tasks that it carries out. By performing this analysis, this report seeks to provide insight into one of the most impactful ransomware attacks in history. Understanding this helps strengthen the cybersecurity field and potentially contributes to preventing future ransomware outbreaks.

2.2 AIMS

This project aims to:

- Perform a comprehensive malware analysis of the chosen sample using various techniques.
- Identify the effects of the malware and investigate how it goes about causing them.
- Gain a deep understanding of how the malware works through static, dynamic and code analysis.
- Document the findings of the malware analysis in this report and provide strong evidence to support them.

3 PROCEDURE AND RESULTS

3.1 OVERVIEW OF PROCEDURE

A safe testing environment was set up to contain the malware sample. A Windows 10 virtual machine is running on VMWare Workstation. The network adapter is set to host only, and guest isolation is turned on to prevent any chance of the malware worming to the virtual machine's host. This also prevents the malware sample from using the internet in any way. A snapshot of the virtual machine is taken before carrying out any actions with the malware so that it can simply be reset to a point before the malware sample was run.

To ensure a strong analysis of the malware sample, a set methodology is followed. First the tester performs static analysis on the malware sample. This is where the malware sample is not running, and the file is analyzed for signs of malicious intent. Various tools are used throughout this process to gather information on the file such as metadata, imported functions, strings, obfuscation techniques and signs of packing.

After this, dynamic analysis was carried out on the malware sample. This is the point where the malware is run, and the tester can analyze in real time how it works. Tools can be used to gather more information and track what tasks the malware is performing, as well as what changes it is making to the host machine in real-time. At this point the tester is analyzing the malware for running executables, making changes to the filesystem, registry modifications, network activity, imported libraries and functions, and changes in the host's memory.

Code analysis is also performed to review the programs source code. This is put into its own separate section because it needs to be done both statically and dynamically. The static sample can be analyzed using disassemblers and hex editor tools. The malware then needs to be run to analyze the code of other aspects of the malware sample, including new processes that the malware creates. This uses the same tools as the static code analysis does, which is why this procedure is grouped up into its own section.

3.2 STATIC ANALYSIS

3.2.1 VirusTotal

The first task to perform in static analysis is to get the checksum hash of the malware sample file and enter it into the "VirusTotal" web application. To do this a PowerShell was opened in the file location of the sample and the "Get-FileHash" command was executed (*view figure 1*).

```
PS C:\Users\user\Desktop\CWMalwareSample\MalwareCW> Get-FileHash .\ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          ED01EBFBC9EB5BBEA545AF4D01BF5F1071661840480439C6E5BABE8E080E41AA       C:\Users\user\Desktop\CWMalwa...
```

Figure 1: Malware sample SHA256 hash

This command returned the SHA256 hash:

ED01EBFBC9EB5BBEA545AF4D01BF5F1071661840480439C6E5BABE8E080E41AA

The hash was entered into VirusTotal (*view figure 2*). 68 out of the 72 security vendors listed on VirusTotal flagged the sample as malicious. VirusTotal also displays the popular threat label "ransomware.wannacry/wannacryptor". At this point the sample is confirmed to be the well-known ransomware "WannaCry".

3.2.2 Strings

Next the sample's strings were investigated. A tool called floss was used to do this as it groups and formats the strings into a more readable format. The tool was executed against the sample and the results were stored in a text file "floss.txt" (*view figure 3*).

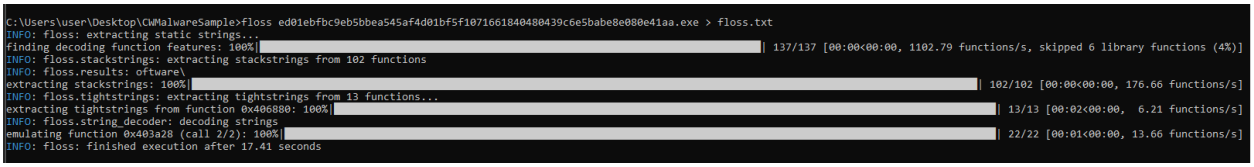


Figure 3: Sample's strings extracted using Floss

Floss manages to automatically find strings encoded in UTF-16LE. These are decoded into a readable format before Floss displays them in the output. This tool finds a lot of strings and many of them are not useful to the investigation therefore, a snippet of the results can be seen (*view figures 4, 5, 6 & 7*). Most of the strings seen here seem to be file extensions and many of them could be seen as work or productivity related e.g. ".docx", ".txt", ".png", ".zip", ".cpp", ".php", ".sql" and ".cmd". These file formats are used for documents, texts, images, compressed directories, software, databases and the windows command prompt. Due to the nature of these files being seen as useful for general work, it is likely that these are the file types that WannaCry targets when encrypting a victim's filesystem.

The floss strings also contain 1 stack string "oftware\" (*view figure 8*). A stack string is a deconstructed string that builds at runtime. This is likely building the string "Software\" which is a common windows registry path which may be used for tasks such as maintaining persistence between restarts and accessing/modifying windows configuration settings. This is further explored in dynamic analysis.

3.2.3 PEStudio

PEStudio is used to statically gather more information on the WannaCry malware sample. The main tab can be seen (*view figure* 9). The software states that the malware was created on Saturday November 20th, 2010, at 9:05:05. This is unlikely because the "EternalBlue" zero-day vulnerability was not leaked until 2017. This means that this is likely intentionally modified to show an incorrect date to attempt to throw analysts off the trail of what exploit the malware may be using. The PEStudio screenshot also shows that there is a very high entropy value present of 7.995, which confirms that the malware sample's code is encrypted. "Ordinary (not packed/compress/encrypted) binary files usually have an entropy of 5; Compressed malware has an entropy of about 6. The closer the entropy gets to 8 (maximum possible entropy), the more likely it is that the code is encrypted", (Morgenstern, 2016).

The indicators tab was then explored (*view figure 10*). PEStudio gave 25 indicators and rated each one on a severity level of 1-3 (1 being the most severe and 3 being the least). 4 of the 25 indicators were flagged as level 1 indicators.

1. Resource size is 3445325b (bytes) - about 3.45mb (megabytes). This is very large compared to most standard program's resource section which likely signifies that something is being hidden in this section.
2. An Embedded file – likely a payload to exploit the device. Non-malicious software does not contain embedded files very often as it is mostly a method of hiding code.
3. A file extension "ransomware wiper" means that the program can perform operations on the host machine's files such as encrypting, editing, or deleting. Most software does not ask for permission to delete or encrypt files; therefore, this is why PEStudio flagged this.
4. Lastly, PEStudio flagged the imports that the malware sample is using. While most programs will use imports, it is the nature of the imports used by the program that is suspicious.

The imports section shows which functions were imported from the libraries that the WannaCry sample makes use of (*view figure 11*). Many of the functions are related to file/directory modification and encryption. WannaCry imports these so it can successfully encrypt the files on the victim's computer and decrypt them should the user pay the ransom.

PEStudio's manifests tab was selected and inspected (*view figure 13*). The main point of interest here is that the application is requesting "asInvoker" privileges. This means that the program is requesting the same privileges that the user has. While this is common in non-malware programs too, this is suspicious when there is no justifiable reason that it would require these permissions.

3.2.4 DiE (Detect it Easy)

A tool called DiE (Detect it easy) was also used to analyze the malware sample further (*view figure 14*). The screenshot shows that the malware sample was written in an old version of Visual Studio and was made using the programming language C++. The same "TimeDateStamp" (time/date of malware creation) was found using DiE and PEStudio. The tool also finds a zipped folder archive where 55.8% (36 files) are encrypted. This is likely to conceal a malicious payload or the malware's configuration files.

This tool also shows a very high entropy value as well as breaking down the malware sample into different regions (*view figure 15*). The tool suggests that the ".rdata" and ".rsrc" sections are packed, while the other sections - "PE Header", ".text" and ".data" - are not.

3.2.5 CFF Explorer

After importing the malware sample into CFF Explorer, the sections headers tab was opened *(view figure 16)*. This breaks down the virtual size and raw sizes of each section so that the tester could manually check for packing using another tool for assurance. This figure also shows that the file starts with a "4D 5A" offset. This is another indication that the file type is a windows executable (".exe" file).

The next CFF Explorer tab that the tester looked at is the import directory tab *(view figure 17).* Here it can be seen that the malware sample imports the following libraries: "KERNEL32.dll", USER32.dll", "ADVAPI32.dll" and "MSVCRT.dll". The "KERNEL32.dll" library is used to handle file operations and memory management. WannaCry likely uses this to create, edit and delete files while running. The "ADVAPI32.dll" library is used in file encryption and methodologies for persisting on the system upon restart. "USER32.dll" is a library that is often used for GUI's (graphical user interfaces); therefore, this is likely used for displaying the ransom note that opens when launching WannaCry. This plays a key role in the ransomware's functionality as without it, the creators of the malware would not be monetarily gaining from the attack. Lastly, the "MSVCRT.dll" library can be used to help with basic memory and

string management. This library plays a core role in supporting the other libraries and functions used by the malware.

3.3 DYNAMIC ANALYSIS

To perform dynamic analysis the tester had to ensure that a clean snapshot was taken of the VM before running the malware sample. This is because upon running it, the system becomes much more difficult to use due to the ransomware popup and many of the files becoming encrypted. This way after running a test or performing a scan, the machine can reset to an older state so that the sample can be analyzed in a different way from the beginning.

3.3.1 File Explorer

The malware sample was extracted into its own directory "MalwareCW" before launching and was the only file in this directory (*view figure 18*). The extracted application was then launched and the "MalwareCW" directory was observed for changes (*view figure 19*). After launching the malware virus, the "MalwareCW" directory is flooded with new files like the "@WanaDecrypter@.exe" which is what runs the ransomware's GUI. It also creates a readme text file with a ransom note (*view figure 20*), and two other executables "taskdl.exe" and "taskse.exe".

There are also two new directories created in this folder, "msg" and "TaskData". The "msg" directory contains many different txt files of the ransomware message but each in a different language. There are 28 in total (*view figure 21*).

Another change that the tester noticed on the now infected VM, is the presence of two new icons on the desktop (*view figures 22, 23 & 24*). The two icons have the same name, "@WanaDecryptor@", except for their file types, one being ".exe", and the other being ".bmp". The ".exe" file is what the victim can use to launch the WannaCry GUI if it is closed (this is talked about in the next section). The ".bmp" file is an image file which is used to replace the desktop wallpaper with a graphic that tells the victim about the ransomware, (WannaCry), that is infecting their machine. The full wallpaper can be seen *(view figure 68).*

A detail was noticed related to the file date. The earliest "date modified" for these newly created process/files was the 9th of May 2017. This is likely much closer to the real creation date of the malware rather than the 2010 date that was being suggested on PEStudio and DiE.

3.3.2 Wana Decrypt0r 2.0 GUI

When the malware was run, a popup GUI (Graphical User Interface) was opened to inform the victims of what was going on and how they can pay the ransom (*view figure 25*). It explains what the malware is doing – encrypting the files on the victim's filesystem – and prompts the victim to pay the ransom to regain access to their files. It states that if the victim does not make the full payment within 3 days, then the price will be doubled and if the victim does not make the payment within 7 days, the filesystem will be completely wiped. The sample even states that it has a function where users can "test" the decrypt function for free to prove that they will in face decrypt the files after payment. It states that this function will decrypt a few of the encrypted files but full payment needs to be made to get the rest. The popup asks the user to send $300 worth of bitcoin to the bitcoin address visible on the popup. The tester also noticed upon performing other tests, the popup window would often place itself in front of

other programs just to get in the way and remind the victim that it is there. While this characteristic is not as persistent as it is in most adware (malware that profits off the victim by forcing ads on screen to trick the user into clicking on them), this happened enough for the tester to actively notice it.

After clicking the "Contact Us" button a small application window opens to allow the victim to enter a message (*view figure 26*). The message has a minimum character count therefore the tester entered random characters and clicked send. This did succeed in any way, seeing as the VM is completely disconnected from the internet.

The "Check Payment" button opened a small window with a loading bar (*view figure 27*). This however, also failed to finish its function due to the lack of internet once again and instead remained stuck at about three quarters progress.

Now the tester investigated the "Decrypt" button. After clicking "start" on the new application window that opened, WannaCry decrypted 10 files at random to prove that it was capable and willing to do so (*view figure 28*). Clicking this button again afterwards did not decrypt anymore files.

3.3.3 Wallet Analysis

The main GUI window of the WannaCry sample displays a bitcoin wallet address.

Bitcoin Wallet Address: 13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb94

This address was analyzed using "blockchain.com" (*view figure 29*). This shows that this one wallet alone received over 20BTC (£1.4 million) and still to this day holds over 0.3BTC (£23,000). The wallet received over 140 transactions, most of them being within the first 5 days (12/5/2017 – 17/5/2017), however, it has received transactions as recent as 10/10/2024.

By tracking the transactions, it can be seen that there are two cases of money being sent out of this wallet two and a half months after the attack, on 3/8/2017 (*view figure 30*). These two transactions were sent to separate wallets and were transactions of 10BTC (£715,000) and 9.6BTC (£687,000) respectively. The money gets sent on a trail of new addresses after this implying that the attackers might have used a cryptocurrency mixer to try and prevent people from tracing the funds.

3.3.4 Process Explorer

Before opening the malware, the tester launched Process Explorer. From here the malware sample was launched and can be seen in Process Explorer (*view figure 31*). The malware sample creates a tree of processes going "@WanaDecryptor@.exe", "taskhsvc.exe" and "conhost.exe". The "taskhsvc.exe" process is likely what carries out the main ransomware actions on the device and therefore the malicious part of the malware. It also had a very normal sounding name when comparing it to other default windows processes, therefore it is likely that it was named this to mask its presence. Lastly, "conhost.exe" is a default windows process which helps to manage console windows (cmd.exe). This is likely used to help interact with the system itself when performing tasks like navigating and encrypting the filesystem.

The "taskhsvc.exe" process handles were investigated (*view figure 32*). One of the handles shows that the IFEO registry key is opened at the registry path: "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options". This registry is very popular amongst malware as it

allows an attacker to attach debuggers to programs. By doing so, attackers can force code execution upon executing or terminating a particular program.

3.3.5 Process Monitor

Process Monitor was used to analyze what operations the malware sample carries out when executed. The VM was reset to a clean snapshot from before the malware was run. Process Monitor was opened, then the malware sample was extracted and executed. Process Monitor shows that the malware sample is performing many concurrent operations at a very high frequency (*view figure 33*). This is likely the malware navigating through and encrypting the machine's filesystem.

A filter was applied to this list to gauge the real number of how many operations are being performed by the sample. The filter is set to exclude any entries whose name do not match the name of the malware sample (*view figure 34*).

After applying this filter, it can be seen that over 340,000 events are operation carried out by the malware sample (*view figure 35*). The reason there are so many events is because the malware is navigating through the entire host machine's filesystem to find files to encrypt.

3.3.6 "taskhsvc.exe" Imports

Seeing as this process is only created after launching the malware, the imports can only be analyzed in a dynamic state. Using Task Manager, the PID of "taskhsvc.exe" was identified to be 5708 (*view figure 36*).

Now that the process's PID had been identified, the sysinternals "listdlls" tool was used on the process (*view figure 37*). This showed lots of imports, some of which were very interesting. It has the same 4 imports that were flagged as interesting when statically analyzing the main sample's imports. However, this process also has a few more cryptographic based imports including "CRYPTBASE.dll" which is one of the most common cryptography functions used in ransomware. It also imports a file that it creates "libevent-2-0-5.dll" along with other files that it created in the same directory: "C:\Users\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\". These functions are all related to how WannaCry would use the Tor network to perform web tasks (like the "Check Payment" or "Contact Us" functions).

3.3.7 Regshot

The virtual machine was reset to a clean state from before the malware was extracted and run. After this Regshot was launched and a snapshot was taken of the systems registry before the sample was launched (*view figure 38*). The malware sample was then extracted and executed, and a second registry snapshot was taken. The Regshot "Compare" function was used to compare the registries (*view figure 39*). This shows that the malware was making modifications to the systems registry as there are 165 total changes. Another snapshot of the registry was taken a few minutes later to see if the malware continued making changes to the registry as time went on. This was once again compared to the first snapshot taken before launching the malware sample (*view figure 40*). The two different registry comparison results were then analysis side by side and the total changes had risen to 178. This showed that further changes had been made after the malware's initial startup and therefore it continued to make changes in the background.

3.3.8 FakeNet-NG

FakeNet-NG was used to set up a fake network to investigate what network functionality the WannaCry sample might contain. Here the tester was looking for instances of it trying to establish connections with other IP addresses and website URLs that it was trying to route the user to. A virtual adapter had to be created for this and was given the static IP address: "192.168.20.10", default gateway: "192.168.20.1", and preferred DNS server: "127.0.0.1". FakeNet-NG was launched (*view figure 41*). Now that FakeNet-NG was up and running, the WannaCry sample was launched, and the traffic was observed (*view figure 42*). As seen in the figure, "@WanaDecryptor@.exe" and "taskhsvc.exe" request to use the DNS server via TCP many times. The tester also interacted with various aspects of the malware sample to see if it would give any different results.

The "Contact Us" button was clicked, a message was entered and sent. FakeNet-NG was inspected (*view figure 43*). Upon clicking "send", FakeNet-NG picked up on quite a few TCP requests. Shortly after, the malware GUI window stopped responding and crashed. This could be related to the message being unable to send due to the lack of internet. The tester also used WireShark to attempt to capture and analyze packets being sent by the malware sample for further network analysis. This, however, did not yield any results.

3.4 CODE ANALYSIS

3.4.1 IDA Disassembler

IDA Free was used to disassemble and analyze the malware sample's machine code (*view figure 44*). This code sample shows the malware sample importing certain functions from KERNEL32.dll library, a library commonly used in malware. Most of the imported functions are related to file modification and reconnaissance (*view figure 45*).

Upon reviewing code snippets, this one seems to be gathering information on the machine that it has infected and performing some directory/file modifications (*view figure 46*).

It can also be seen that the malware is importing functions from the MSVCRT.dll library (*view figure 47*). This is also a library whose functions are related to string manipulation, file modification and memory management. It is likely that the malware sample is using these default windows libraries (KERNEL32.dll and MSVCRT.dll) to perform malicious actions while making it harder to detect it.

The sample also imports USER32.dll functions (*view figure 48*). This library is commonly used for contributing to making GUI's work, but it can also be used to store data in a buffer. This means, in the context of WannaCry, it is likely used for string construction and obfuscation as we know this sample partakes in these stealth techniques.

Now the tester loaded the "@WanaCry@.exe" process into IDA and searched the strings for "taskhsvc.exe" (*view figure 49*). After finding it (*view figure 50*), the tester jumped to its source code to check if it is being launched by this parent process, or a different one. It can be seen that the "taskhsvc.exe" process is being launched by this parent process (*view figure 51*). The code snippet shows the parent process "@WanaDecryptor@.exe" process launching the child process "taskhsvc.exe".

A snippet of code shows some cryptography functions that were used in the "taskhsvc.exe" process (*view figure 52*). It shows functions to create an encryption key, decrypt data, encrypt data, destroy the

created encryption key, import an encryption key and lastly a function used for connecting to the encryption provider.

3.4.2 Ollydbg

Ollydbg was used to further analyze the "taskhsvc.exe" machine code. This received mostly the same results as the IDA investigation, therefore, previously discussed points will be ignored here (*view figures* 53, 54, 55, 56 & 57).

The hardcoded wallet address was found in this snippet of code (*view figure 58*). Underneath it was two other strings of the exact same length, which means they are likely other cryptocurrency wallets attached to the WannaCry ransom. To test this, the addresses were looked up on "blockchain.com". Both were real bitcoin addresses and showed that they received their first payment on the 12th of May 2017, the same day that WannaCry was launched (*view figures 66 & 67*). They also both received many payments within the first 7 days, aligning to WannaCry's ransom technique of threatening to delete the files after a week of no payment. This confirmed that they were alternative wallets for collecting ransom money.

The executable modules tab shows the imports used in this process (*view figure 59*). Many of the same common malicious libraries that have been seen previously are present. There are also a few other cryptography-based libraries here like "CRYPTBASE.dll" and "bcryptPrimitives.dll". Lastly, the malware can be seen interacting with registry keys/ directories in some way (*view figure 60*).

3.4.3 010 Editor

A hex editor tool called 010 Editor was used to investigate some of the files created in the MalwareCW directory (where the sample was extracted and launched). The full MalwareCW directory can be seen (*view figure 19*). The first 3 files of interest were "00000000.eky", "00000000.pky" and "00000000.res" (*view figure 61*).

These are custom filetypes and in this context are related to RSA encryption. This became more apparent upon inspecting "00000000.pky" as it begins with "RSA1" (*view figure 62*). This file is the RSA encryption public key.

The "00000000.eky" file is the RSA encryption private key and can be seen (*view figure 63*).

This leaves the "00000000.res" file which is less clear what it actually does as it does not seem to have much data in it (*view figure 64*). Despite this, the name implies that it is likely related to the other files involved in RSA encryption.

010 Editor was used to open some of the other files in this directory as well, the most intriguing being "c.wnry" (*view figure 65*). Analyzing this in the hex editor showed the hardcoded bitcoin wallet address that is being used by this WannaCry sample. It also showed a few Tor website URL addresses. These are seemingly random strings ending with the ".onion" suffix, which is usually used to hide services on the Tor network. These are likely related to anonymously receiving payments and receiving and sending messages with victims through the "Contact Us" function. The file also shows a link to the website to install the tor browser which is likely performed by the malware sample if it is given an internet connection.

4 DISCUSSION

4.1 GENERAL DISCUSSION

After analyzing the malware sample, it was very quickly found that it was the well-known ransomware "WannaCry". There are many different variations of WannaCry, therefore, simply knowing what it is does not mean that the tester can fully know how it works based on its previous documentation. During the initial outbreak of WannaCry, many malware experts began to perform malware analysis on the ransomware to gather as much information as possible. In this stage it is crucial to gather as much information as possible on the sample so that it can be combatted, and the damage can be controlled.

Despite having found imported functions, which are usually used for persistence, in the malware sample's machine code, this sample does not seem to have persistence features in place. This means that the malware can simply be closed and will not launch itself when the victim restarts their infected machine. This makes sense as after running once, the malware focuses on encrypting the victim's filesystem, meaning the damage is already done and there is no need to attempt to remain on the victim's machine. It is now on the victim to choose to open the ransomware GUI and make the payment to retrieve their files.

Analyzing the attached cryptocurrency wallet that was hardcoded into the WannaCry malware sample showed that the ransomware received most of its profits within the first week of it being live. WannaCry's launch is publicly known to be 12/5/2017, which perfectly lines up with when the first transaction was made. The reason it made so much within the first week would be because of the imminent threat of deleting all files on the victims file system after the 7th day of not receiving a payment. WannaCry is known to have 3 different hard-coded cryptocurrency addresses, which is why the address analyzed in this sample only makes up £1.4 million of WannaCry's total profits. It is important to note that all 3 of the cryptocurrency wallet addresses were found in the machine code of this sample, but only one of them was formally offered to the victim.

One of WannaCry's most well-known features is its ability to worm to other devices on the network and infect them. It used a windows zero-day vulnerability called "EternalBlue" to do this. When performing network analysis on this variant of WannaCry using FakeNet-NG and Wireshark, it was found that this variant may not have worming capabilities. This was decided by the fact that the sample never made any requests to any domains or other devices. It did seem to use TCP to ping the DNS server, which is likely a form of reconnaissance, looking for other devices on the network. Therefore, it is fair to assume that maybe the malware sample would have acted differently if it had managed to find another device on the network and may then show evidence of worming functionality.

4.2 COUNTERMEASURES

There are plenty of methods rules that can be followed to prevent malware from being an issue. The first method would be to prevent it from ever infecting a device. This can be done by being educated in ways it can spread, keeping devices and services up to date, and having active anti-malware software to detect unusual files or behavior. “Microsoft were made aware of a potential attack 12 months prior to the attack and released a security patch to be installed on all electronic devices that ran Windows”, (National Health Service, 2023). Microsoft rolled out a patch to this on the 14th of March 2017, almost two full months before the attack (Microsoft, 2017). Despite this, many large companies, like the NHS, did not take Microsoft’s warning seriously and did not rush to install the critical security patch, which resulted in WannaCry’s unfortunate success.

However, preventing malware from infecting machines is not always possible and therefore there are also ways to counter malware after a machine has been infected. In the context of ransomware, keeping backups of work on external devices would allow victims to regain their files without having to pay the ransom, the only downside is the need to keep this backup up to date. It may also be necessary to have this disconnected from the network as if the malware is capable of worming, then the machine holding the backup could also be infected. A firewall is also essential to ensure that the malware is not performing any malicious network activity after infecting a machine. This could also contribute to helping quarantine the malware alongside the disconnected network.

4.3 FUTURE WORK

Given more time, network analysis would have been further investigated. While it seemed that this particular WannaCry sample did not have worming capabilities, this could be due to the fact that it did not find any other devices on the network. A separate VM could be created and placed on an isolated network alongside the infected machine. This would then likely be detected by the malware sample when performing TCP DNS port probing and may then act further than what was seen in the tests documented in this report. The network functions on the malware GUI could also be further investigated. This includes the “Check Payment” and “Contact Us” functions. These could very likely be related to the “.onion” addresses found in the “c.wnry” file.

5 REFERENCES

Blockchain.com. (n.d.). *115p7-6LrLn*. Retrieved from blockchain.com: https://www.blockchain.com/explorer/addresses/btc/115p7UMMngoj1pMvkpHijcRdfJNXj6LrLn

Blockchain.com. (n.d.). *12t9Y-r6SMw*. Retrieved from blockchain.com: https://www.blockchain.com/explorer/addresses/btc/12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw

Blockchain.com. (n.d.). *13AM4-aEb94*. Retrieved from blockchain.com: https://www.blockchain.com/explorer/addresses/btc/13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb94

Hughes, O. (2018, October 12). *Government puts cost of WannaCry to NHS at £92m*. Retrieved from DigitalHealth: https://www.digitalhealth.net/2018/10/dhsc-puts-cost-wannacry-nhs-92m/#:~:text=The%20Department%20of%20Health%20and,£72m%20in%20the%20aftermath.

Kaspersky. (n.d.). *What is WannaCry ransomware?* Retrieved from Kaspersky: https://www.kaspersky.com/resource-center/threats/ransomware-wannacry

Microsoft. (2017, March 14). *Microsoft Security Bulletin MS17-010 - Critical*. Retrieved from microsoft.com: https://learn.microsoft.com/en-us/security-updates/securitybulletins/2017/ms17-010

Morgenstern, T. (2016, December 15). *Malware Terms for Non-Techies – Code Entropy*. Retrieved from cyberbit: https://www.cyberbit.com/endpoint-security/malware-terms-code-entropy/#:~:text=Ordinary%20(not%20packed%2Fcompress%2F,that%20the%20code%20is%20encrypted.

National Health Service. (2023, April 21). *NHS England business continuity management toolkit case study: WannaCry attack*. Retrieved from NHS: https://www.england.nhs.uk/long-read/case-study-wannacry-attack/

APPENDICES

APPENDIX A

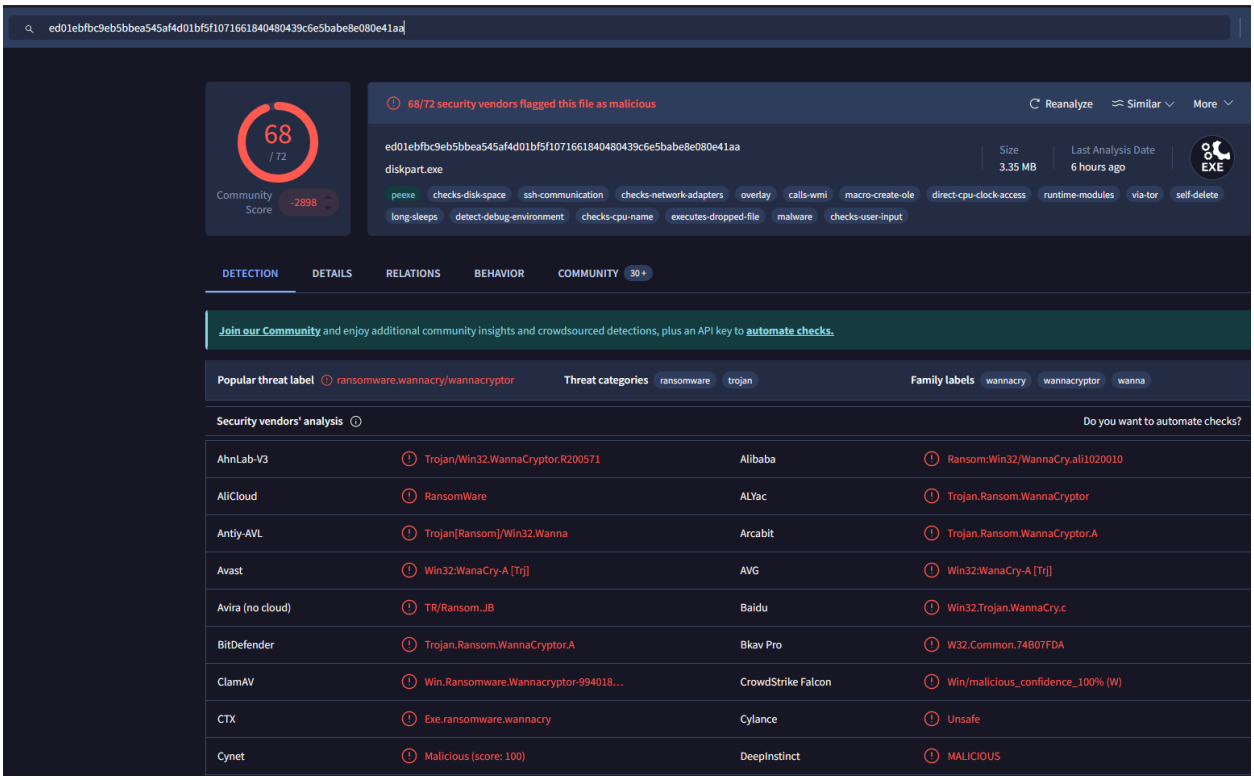


Figure 2: Hash searched on VirusTotal

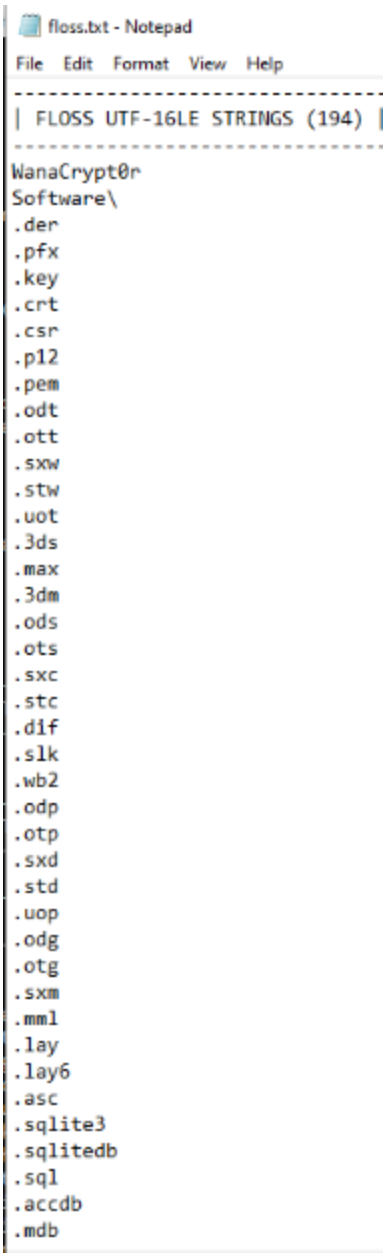


Figure 4: Floss.txt (1/5)

```
.odb
.frm
.myd
.myi
.ibd
.mdf
.ldf
.sln
.suo
.cpp
.pas
.asm
.cmd
.bat
.ps1
.vbs
.dip
.dch
.sch
.brd
.jsp
.php
.asp
.java
.jar
.class
.mp3
.wav
.swf
.fla
.wmv
.mpg
.vob
.mpeg
.asf
.avi
.mov
.mp4
.3gp
.mkv
.3g2
.flv
.wma
.mid
```

Figure 5: Floss.txt (2/5)

Figure 6: Floss.txt (3/5)

```
.rtf
.csv
.txt
.vsdx
.vsd
.edb
.eml
.msg
.ost
.pst
.potm
.potx
.ppam
.ppsx
.ppsm
.pps
.pot
.pptm
.pptx
.ppt
.xltm
.xltx
.xlc
.xlm
.xlt
.xlw
.xlsb
.xlsm
.xlsx
.xls
.dotx
.dotm
.dot
.docm
.docb
.docx
.doc
%s\%s
%s\Intel
%s\ProgramData
VS_VERSION_INFO
StringFileInfo
040904B0
CompanyName
```

Figure 7: Floss.txt (4/5)

```
Microsoft Corporation
FileDescription
DiskPart
FileVersion
6.1.7601.17514 (win7sp1_rtm.101119-1850)
InternalName
diskpart.exe
LegalCopyright
 Microsoft Corporation. All rights reserved.
OriginalFilename
diskpart.exe
ProductName
Microsoft
 Windows
 Operating System
ProductVersion
6.1.7601.17514
VarFileInfo
Translation

---------------------------
| FLOSS STACK STRINGS (1) |
---------------------------
oftware\

---------------------------
| FLOSS TIGHT STRINGS (0) |
---------------------------

-----------------------------
| FLOSS DECODED STRINGS (0) |
-----------------------------
```

Figure 8: Floss.txt (5/5)

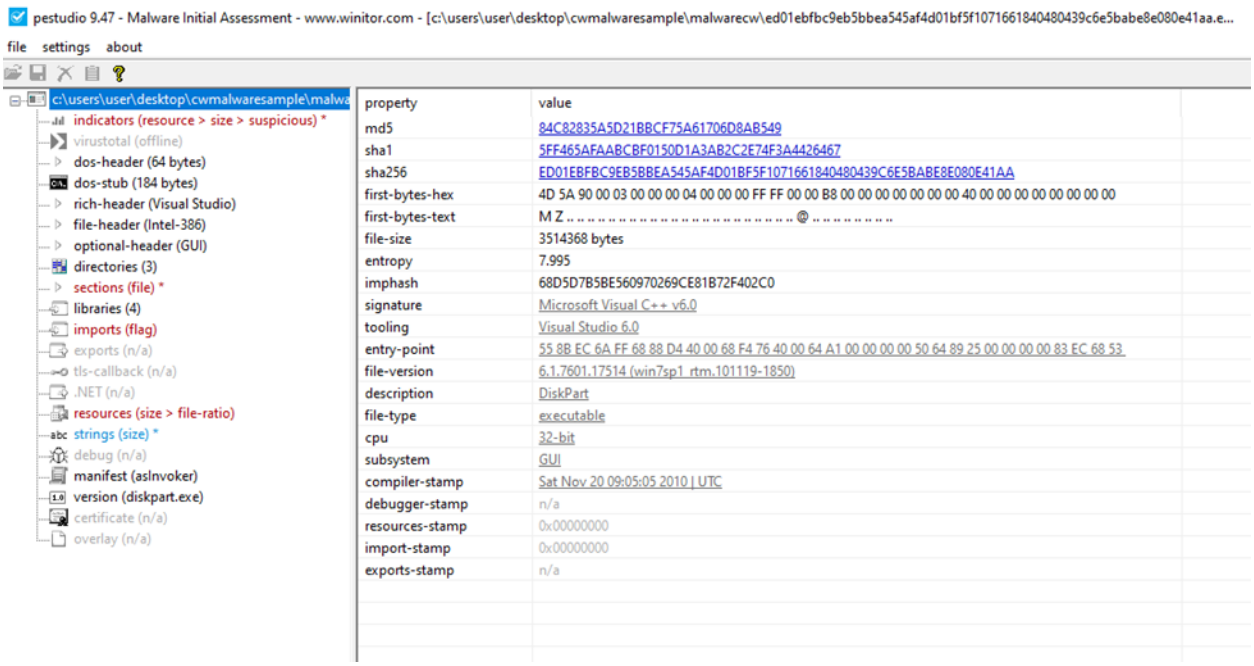


Figure 9: PEStudio main tab

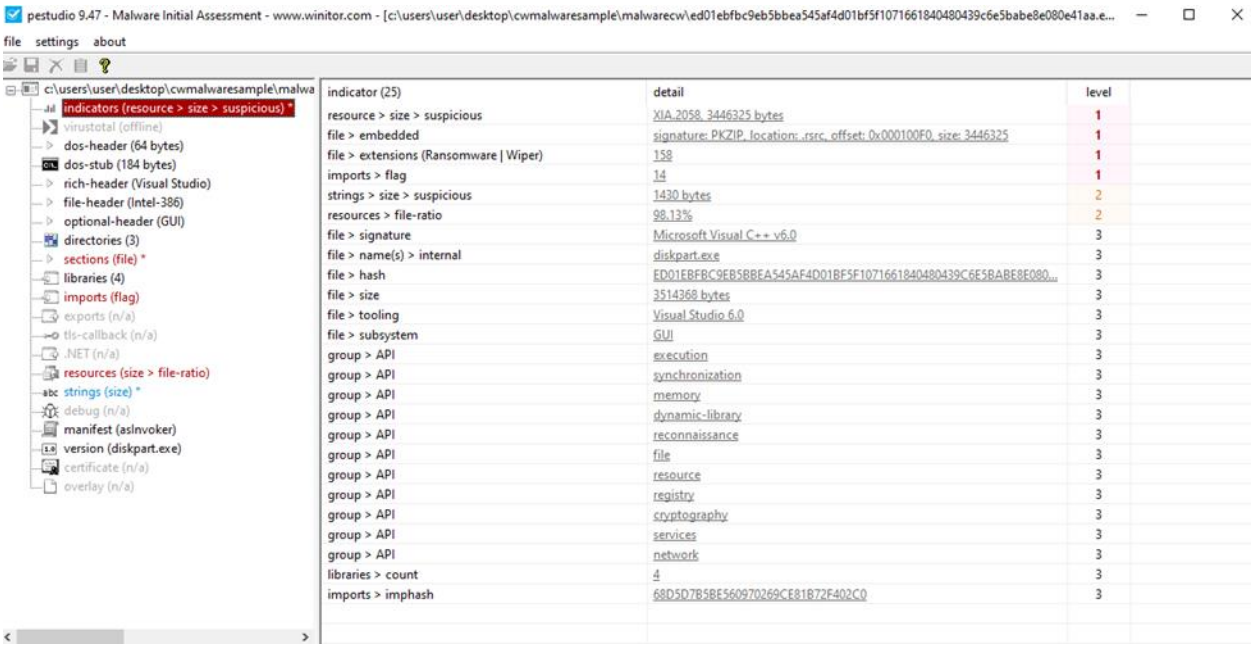


Figure 10: PEStudio indicators tab

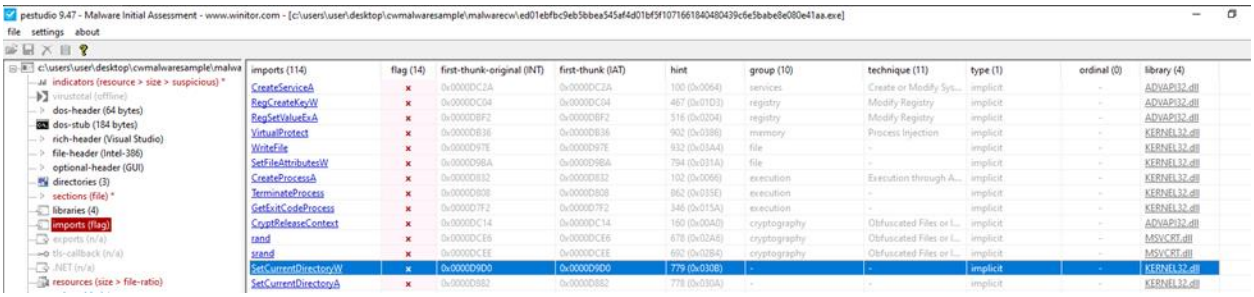


Figure 11: PEStudio Imports tab

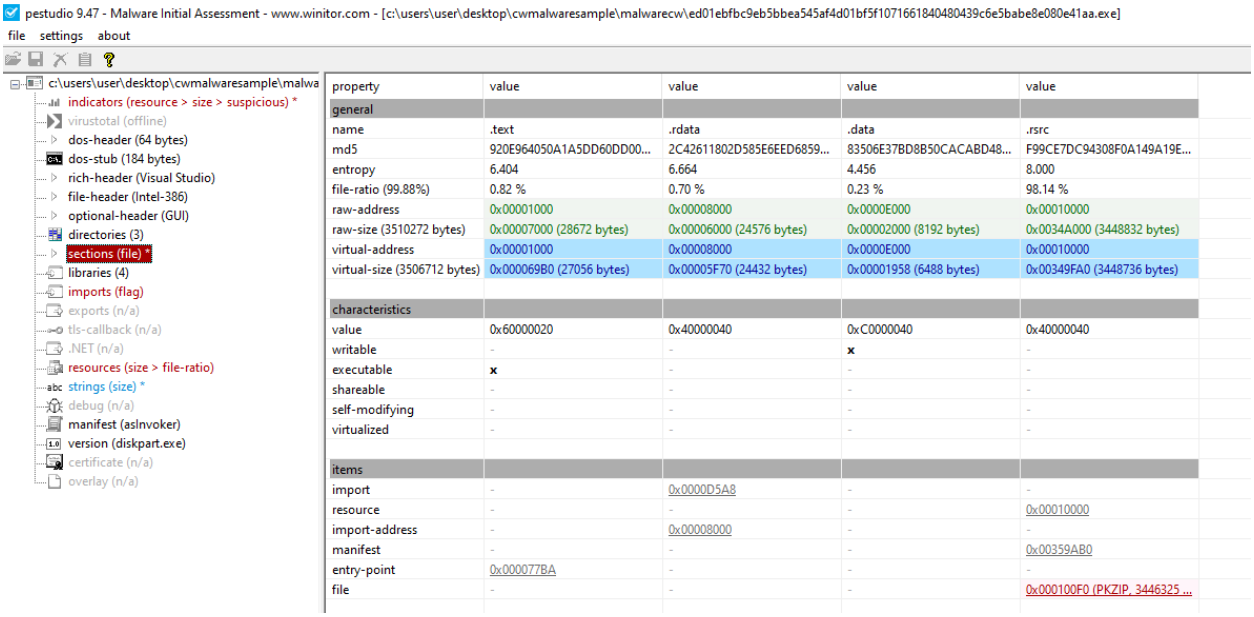


Figure 12: PEStudio sections tab

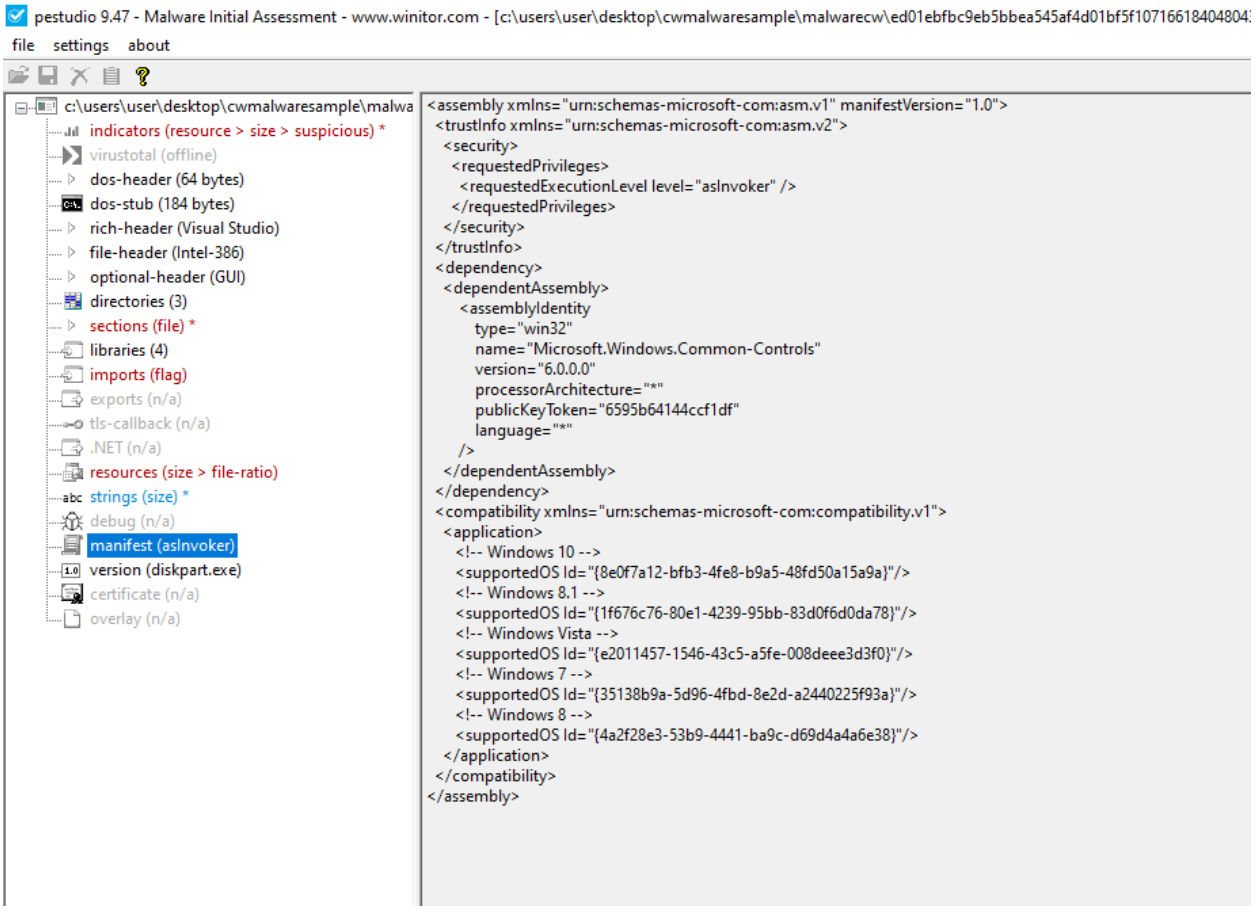


Figure 13: PEStudio manifest tab

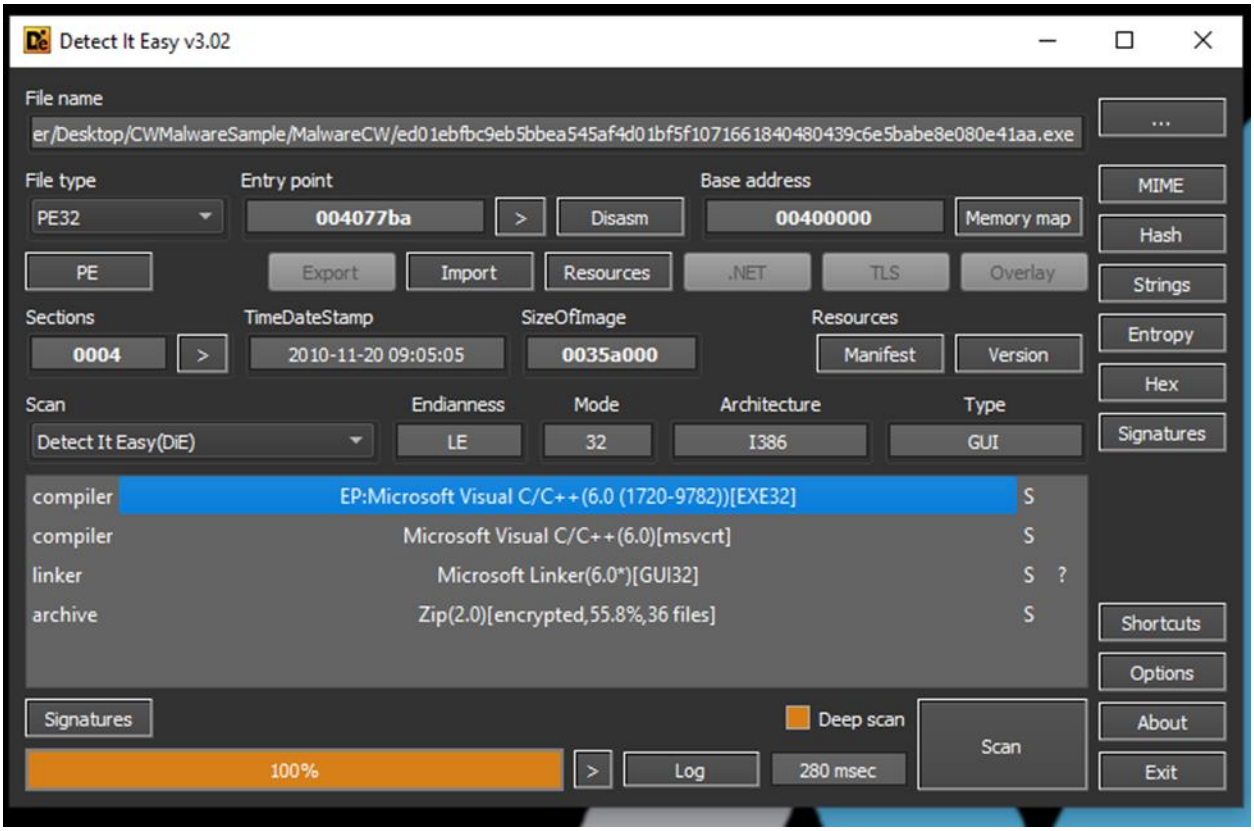


Figure 14: DiE main tab

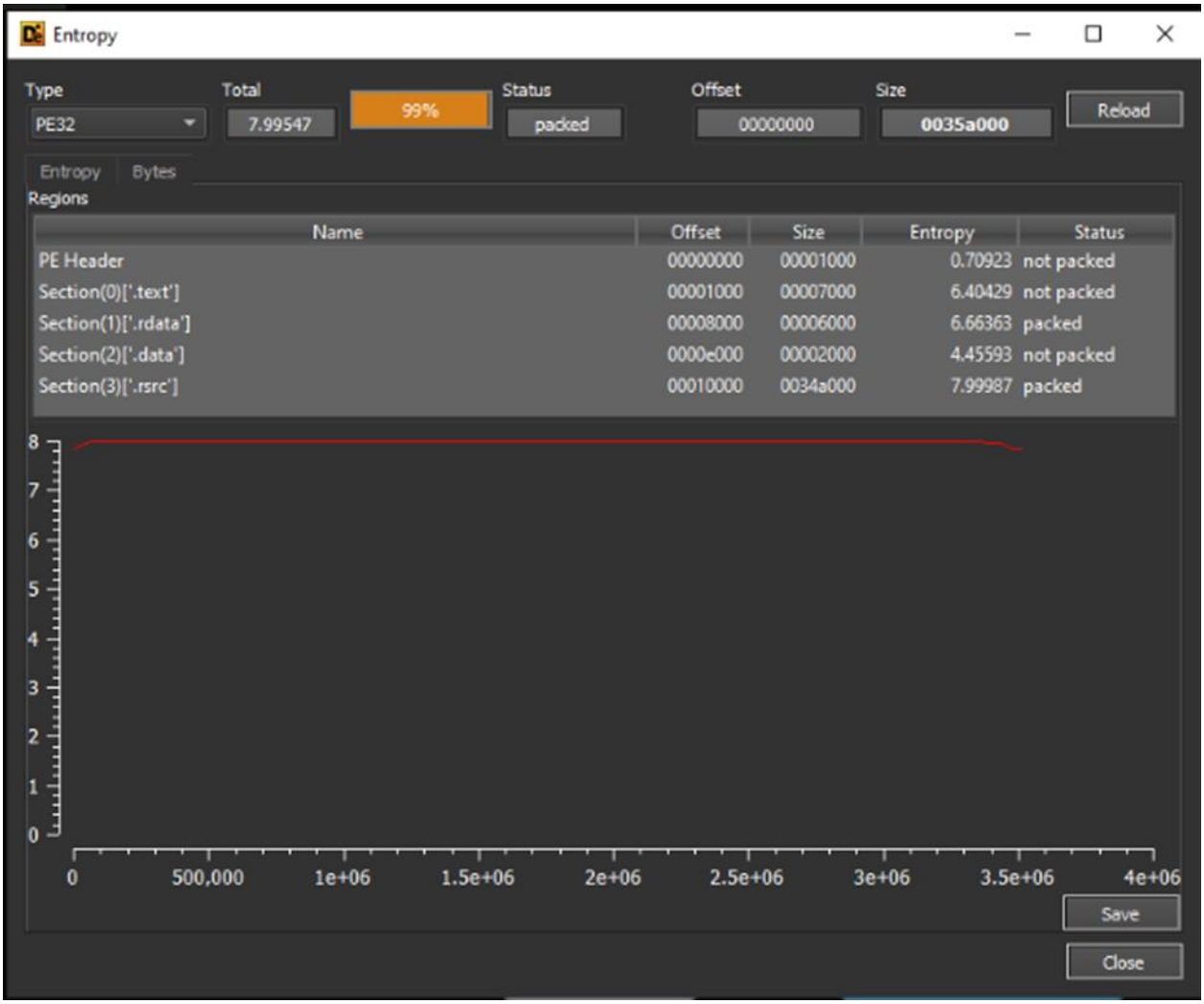


Figure 15: DiE entropy tab

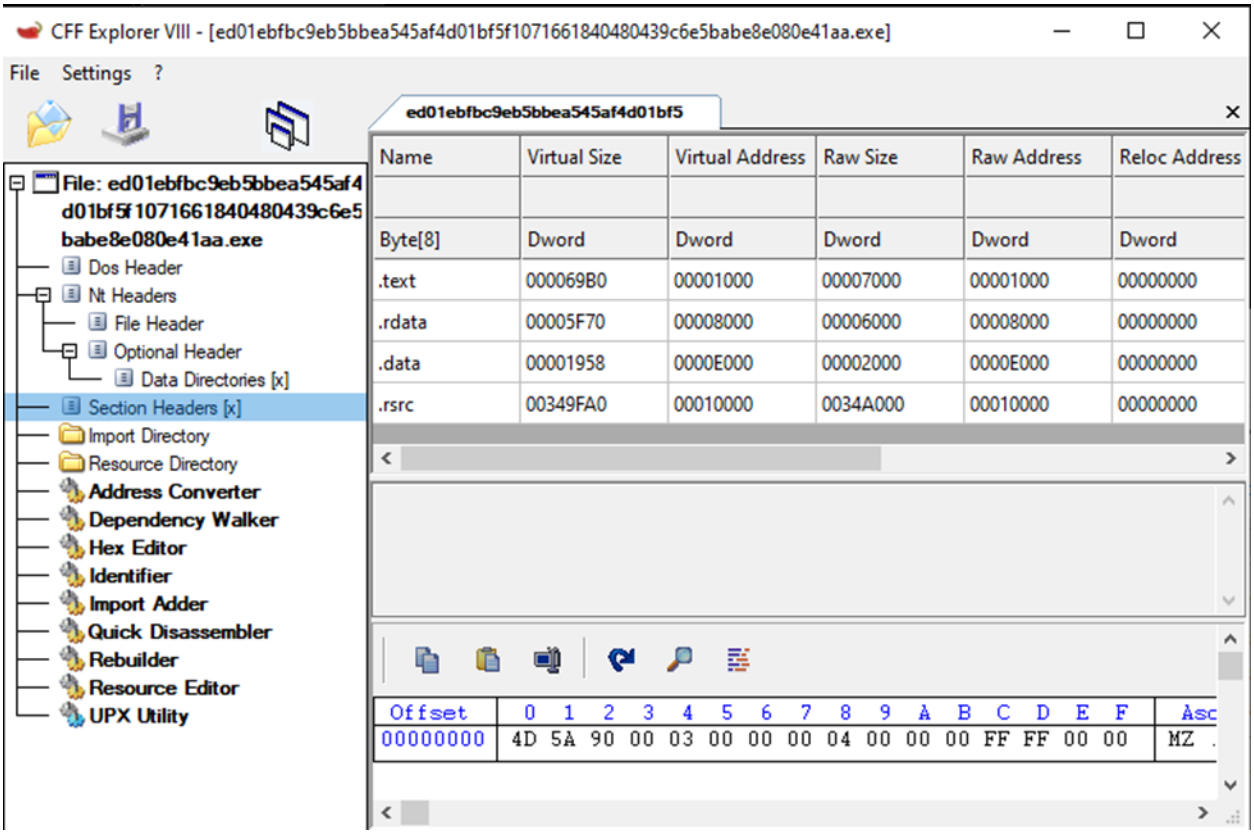


Figure 16: CFF Explorer section headers tab

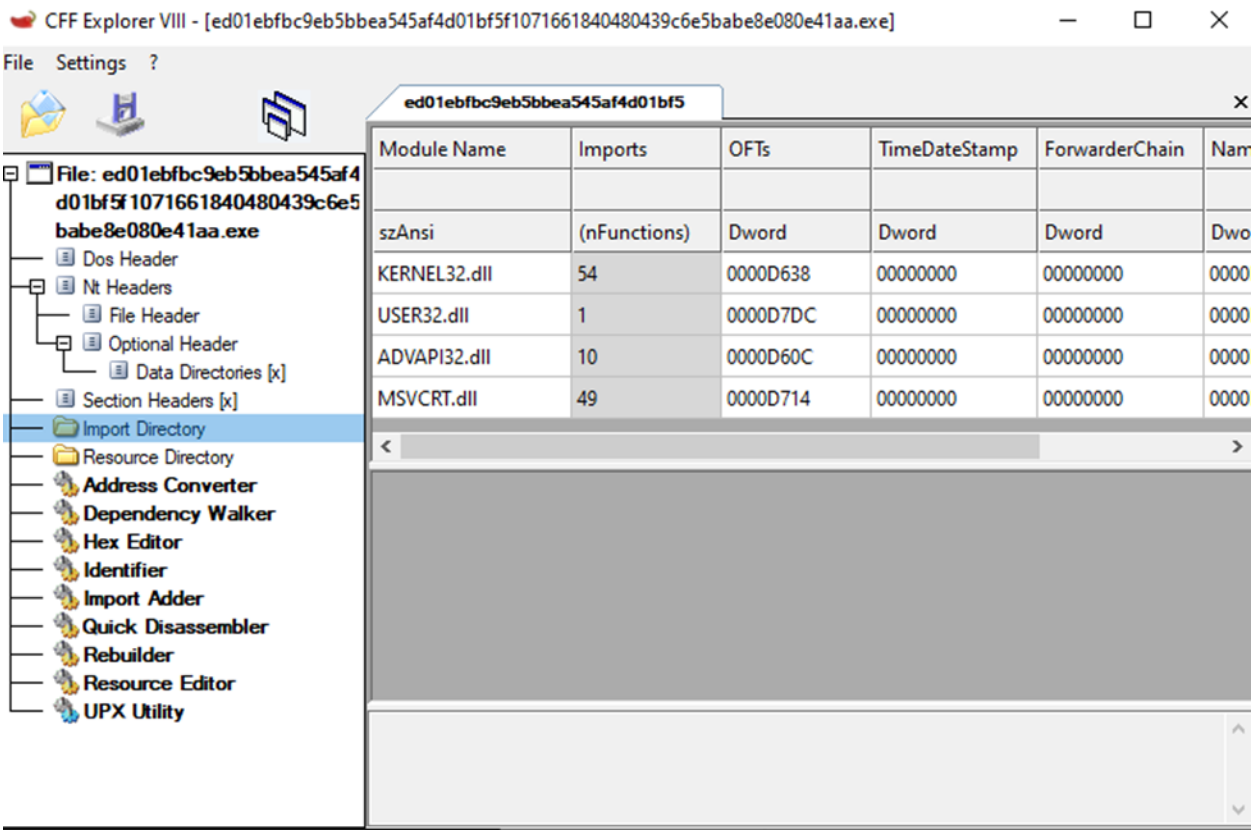


Figure 17: CFF Explorer import directory tab

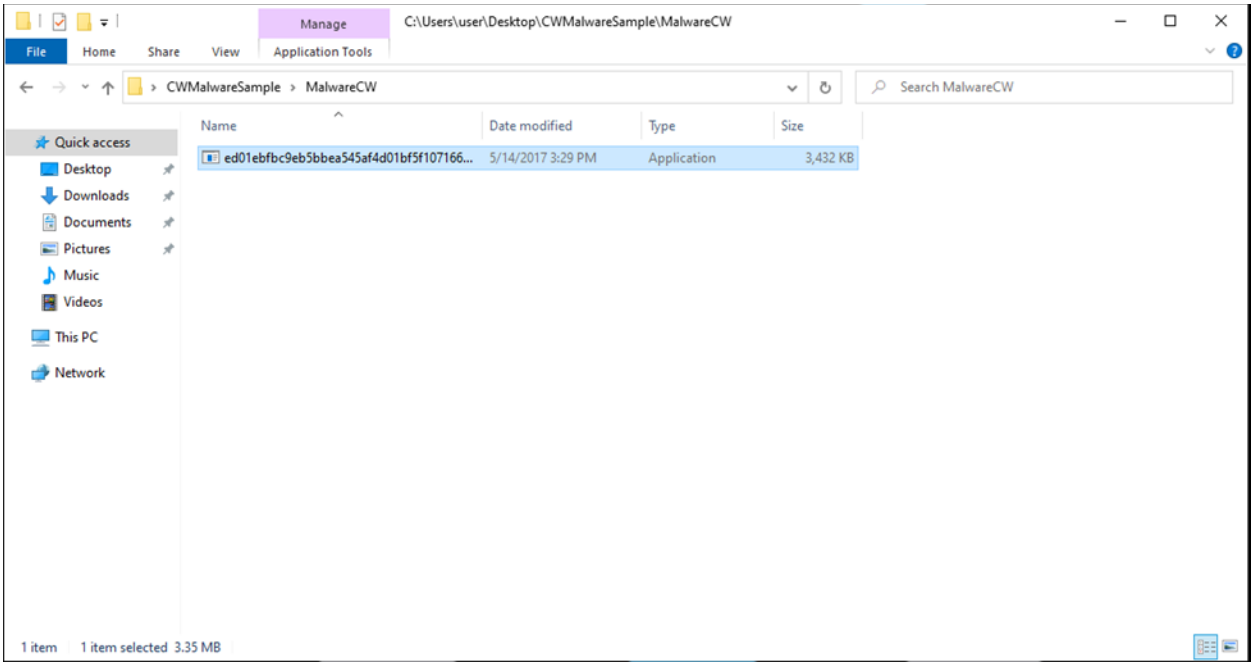


Figure 18: File explorer "MalwareCW" directory

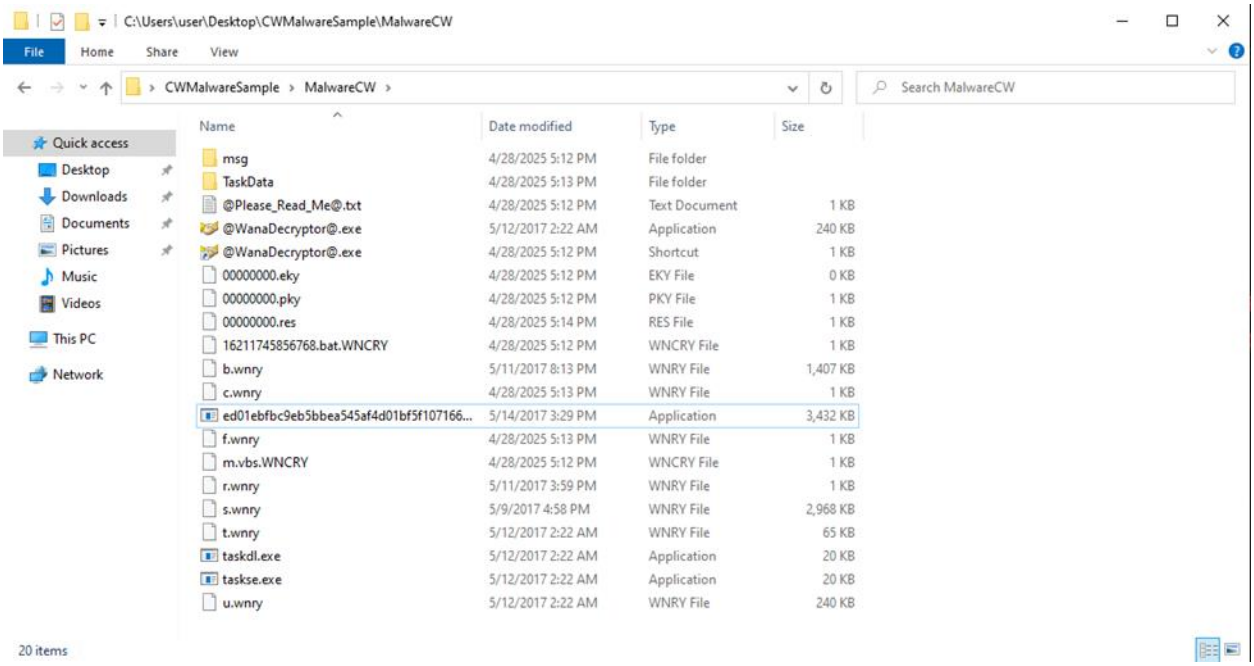


Figure 19: File explorer "MalwareCW" directory post malware launch

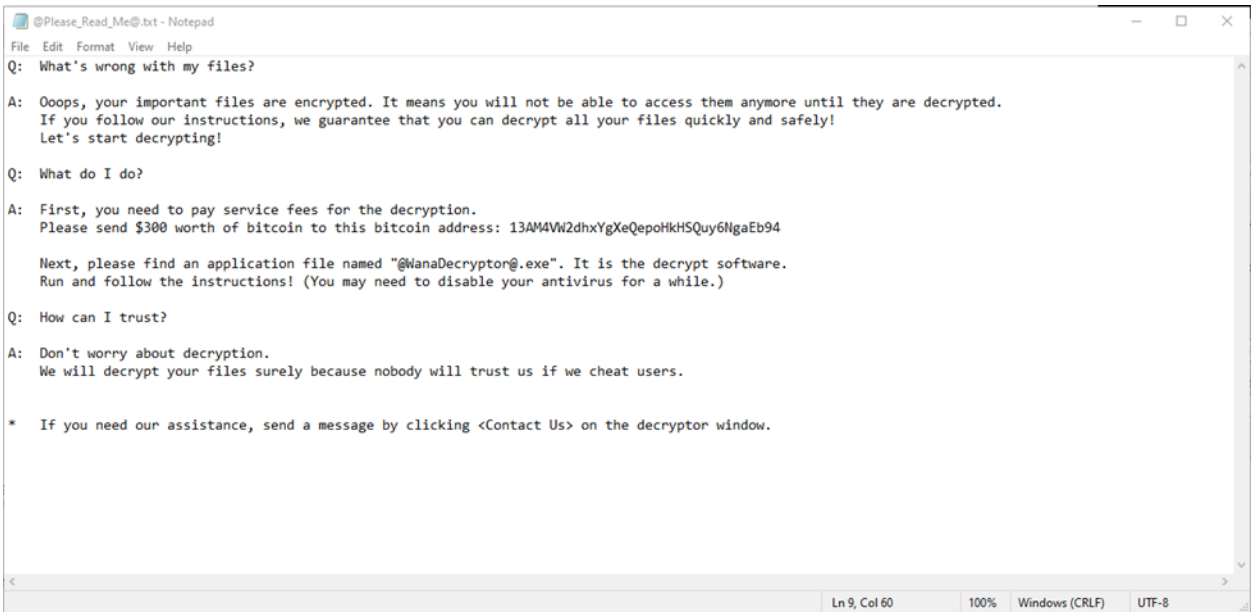


Figure 20: WannaCry ransom note README text file

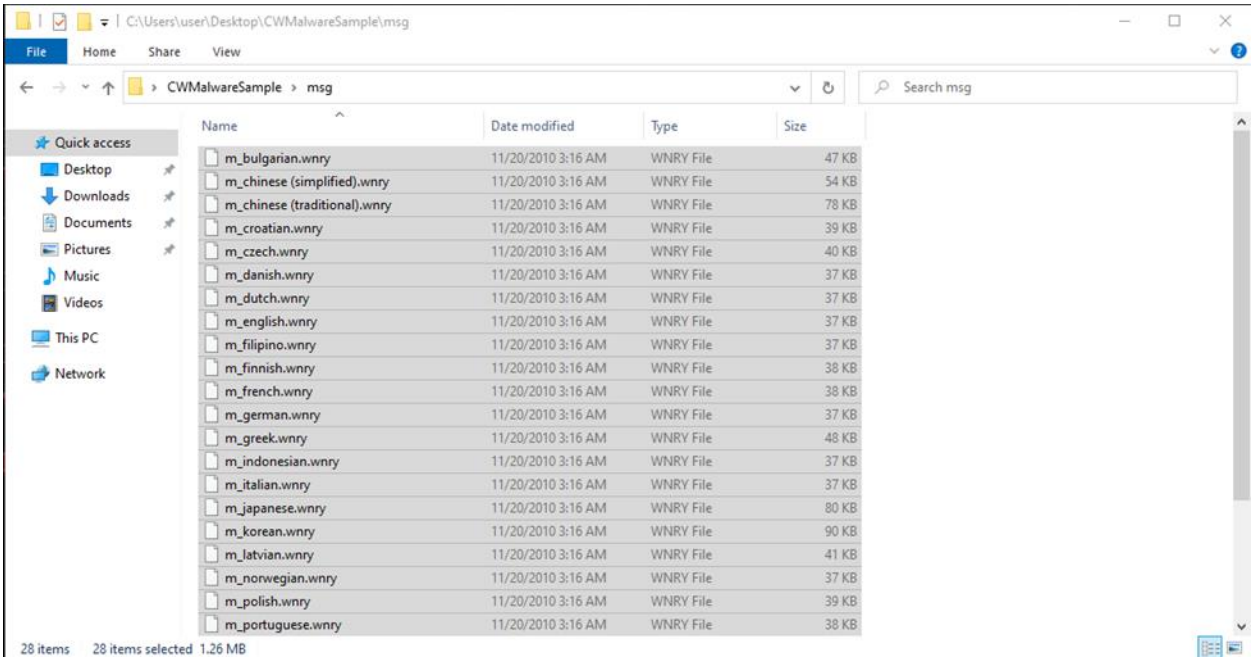


Figure 21: File Explorer "msg" directory containing the ransom note in many different languages

Figure 22: Desktop (1/3)

Figure 23: Desktop (2/3)

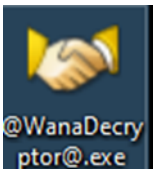


Figure 24: Desktop (3/3)

Figure 25: WannaCry GUI

Figure 26: WannaCry GUI contact us

Figure 27: WannaCry GUI check payment

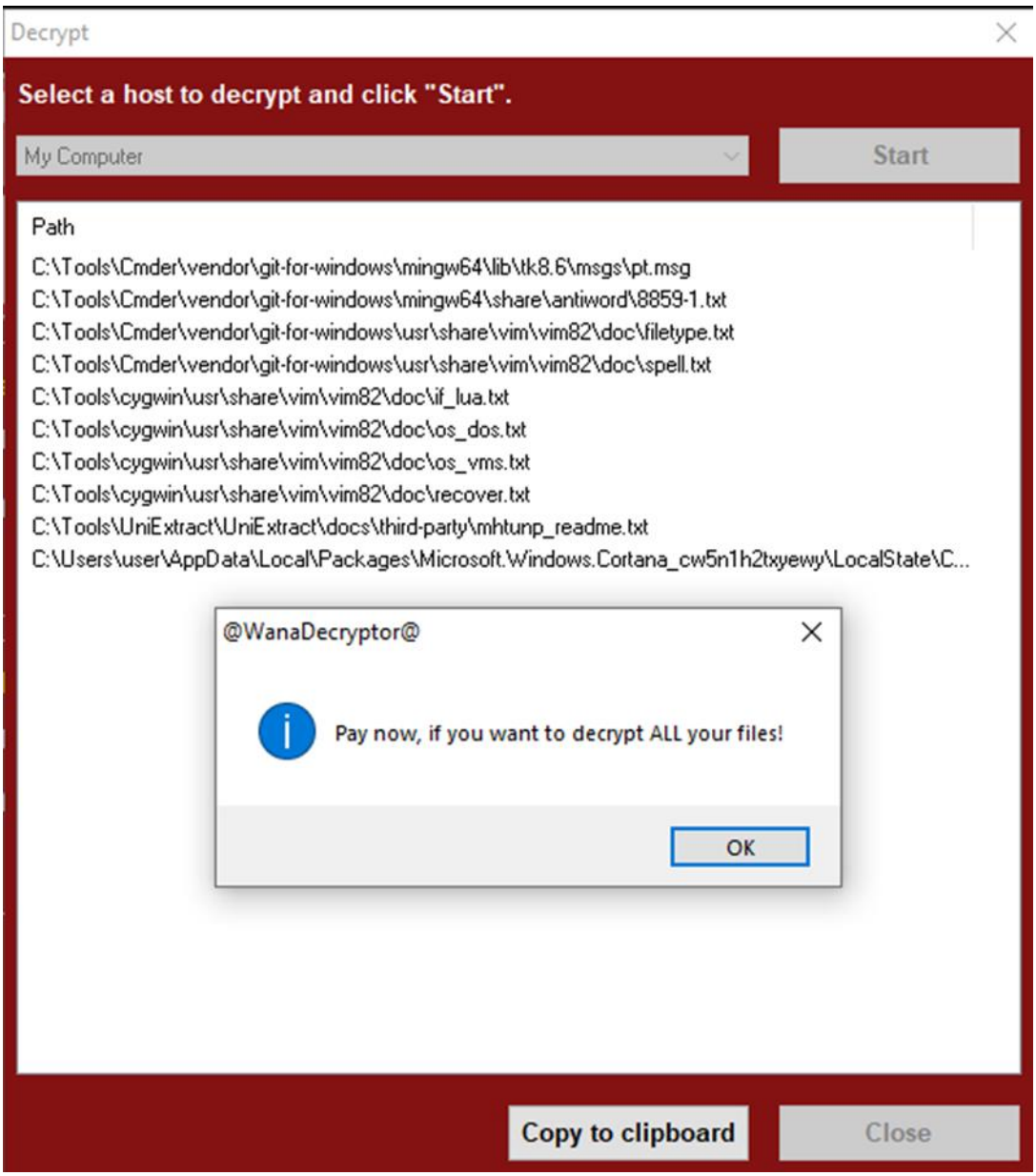


Figure 28: WannaCry GUI decrypt

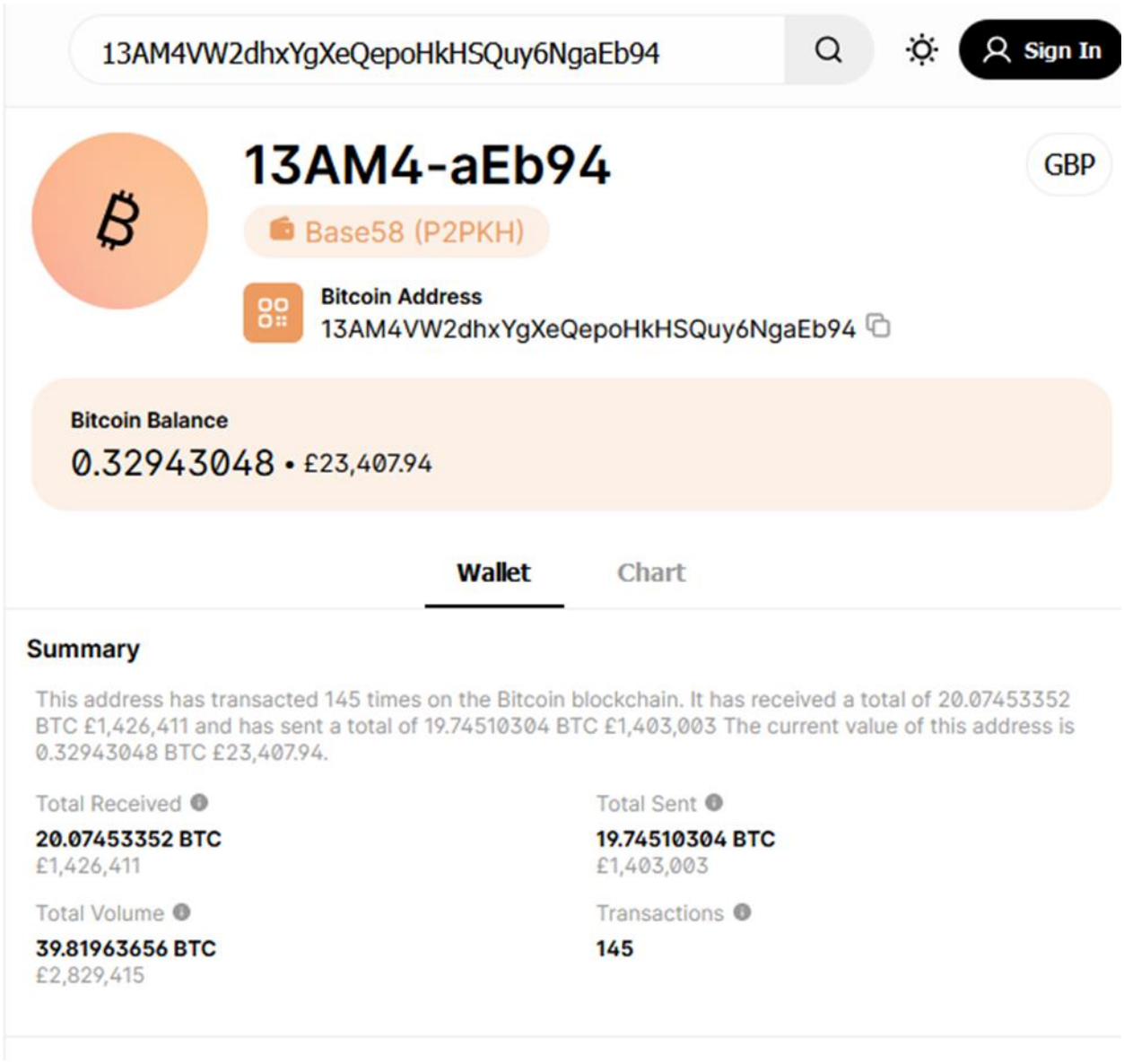


Figure 29: Bitcoin wallet address (Blockchain.com, n.d.)

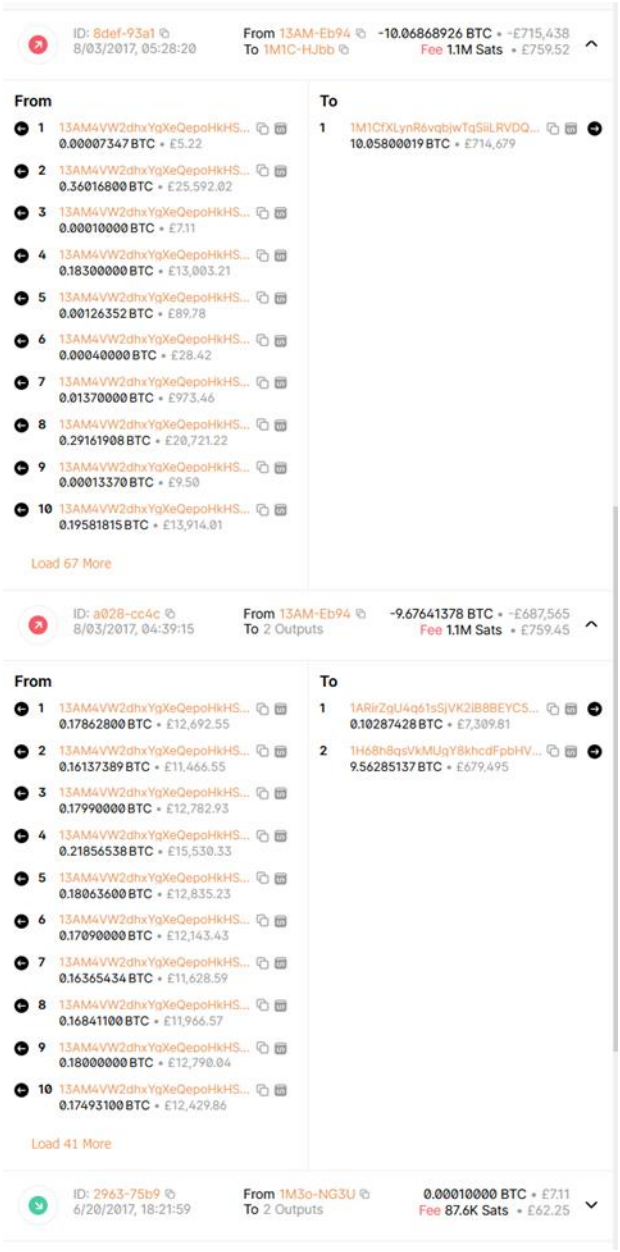
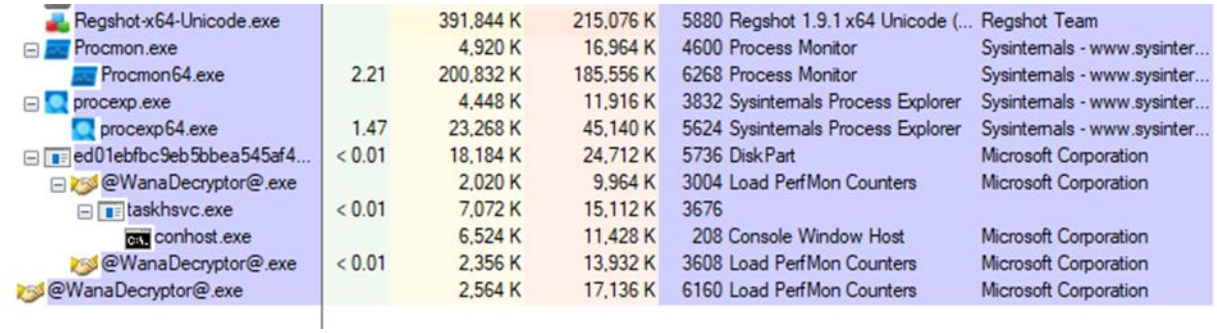


Figure 30: Bitcoin wallet address's out payments (Blockchain.com, n.d.)



Regshot-x64-Unicode.exe		391,844 K	215,076 K	5880	Regshot 1.9.1 x64 Unicode (...	Regshot Team
Procmon.exe		4,920 K	16,964 K	4600	Process Monitor	Sysinternals - www.sysinter...
Procmon64.exe	2.21	200,832 K	185,556 K	6268	Process Monitor	Sysinternals - www.sysinter...
procexp.exe		4,448 K	11,916 K	3832	Sysinternals Process Explorer	Sysinternals - www.sysinter...
procexp64.exe	1.47	23,268 K	45,140 K	5624	Sysinternals Process Explorer	Sysinternals - www.sysinter...
ed01ebfbc9eb5bbea545af4...	< 0.01	18,184 K	24,712 K	5736	DiskPart	Microsoft Corporation
@WanaDecryptor@.exe		2,020 K	9,964 K	3004	Load PerfMon Counters	Microsoft Corporation
taskhsvc.exe	< 0.01	7,072 K	15,112 K	3676		
conhost.exe		6,524 K	11,428 K	208	Console Window Host	Microsoft Corporation
@WanaDecryptor@.exe	< 0.01	2,356 K	13,932 K	3608	Load PerfMon Counters	Microsoft Corporation
@WanaDecryptor@.exe		2,564 K	17,136 K	6160	Load PerfMon Counters	Microsoft Corporation

Figure 31: Process Explorer

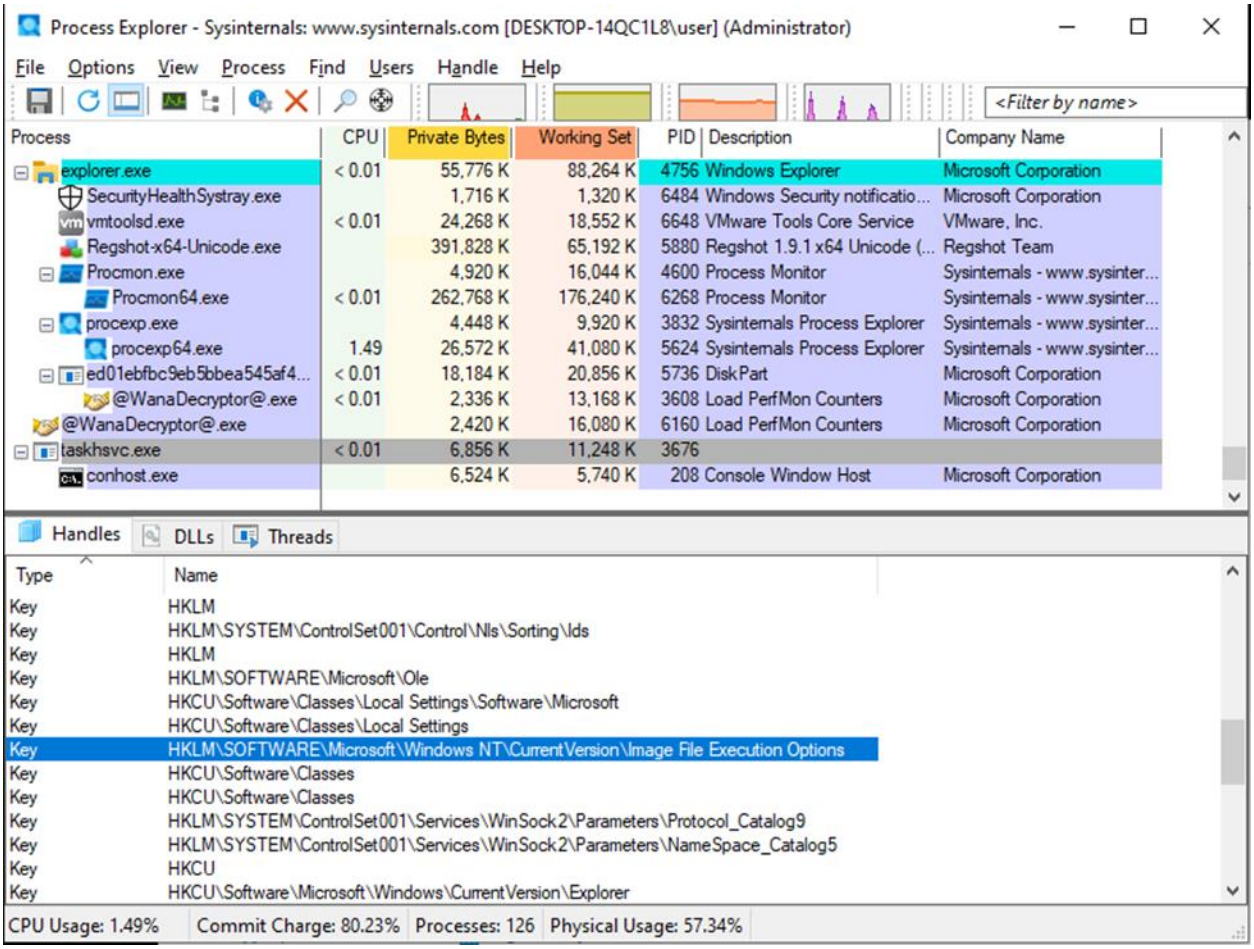


Figure 32: Process Explorer handles tab

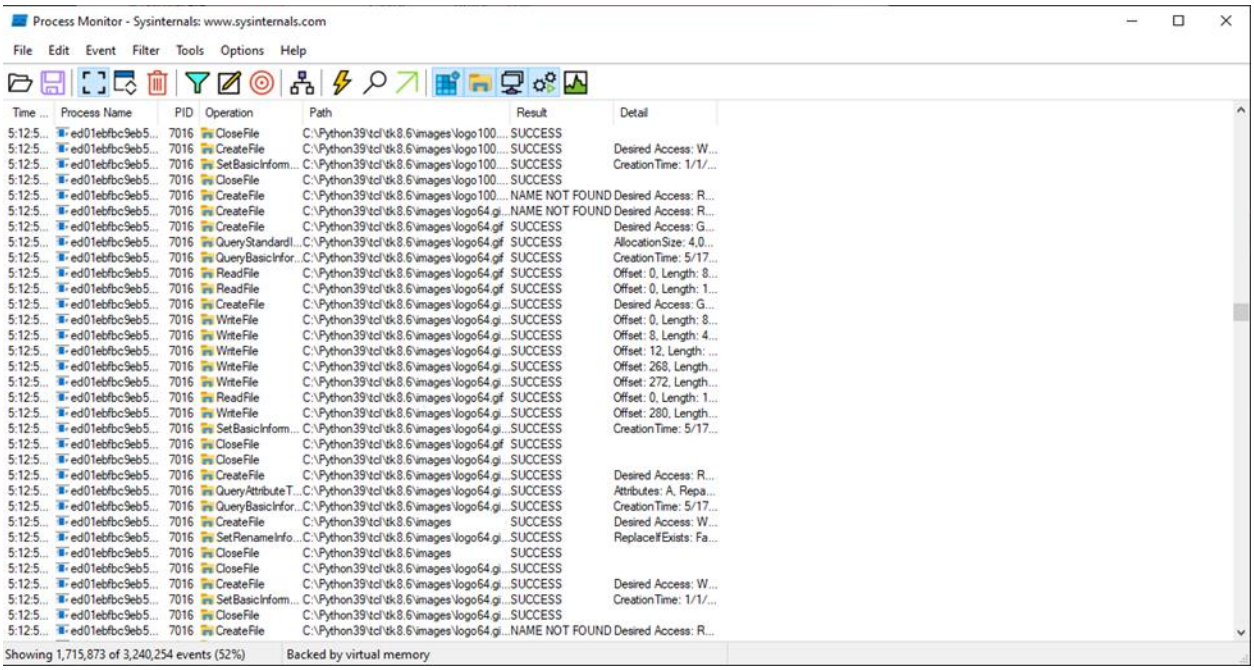


Figure 33: Process Monitor

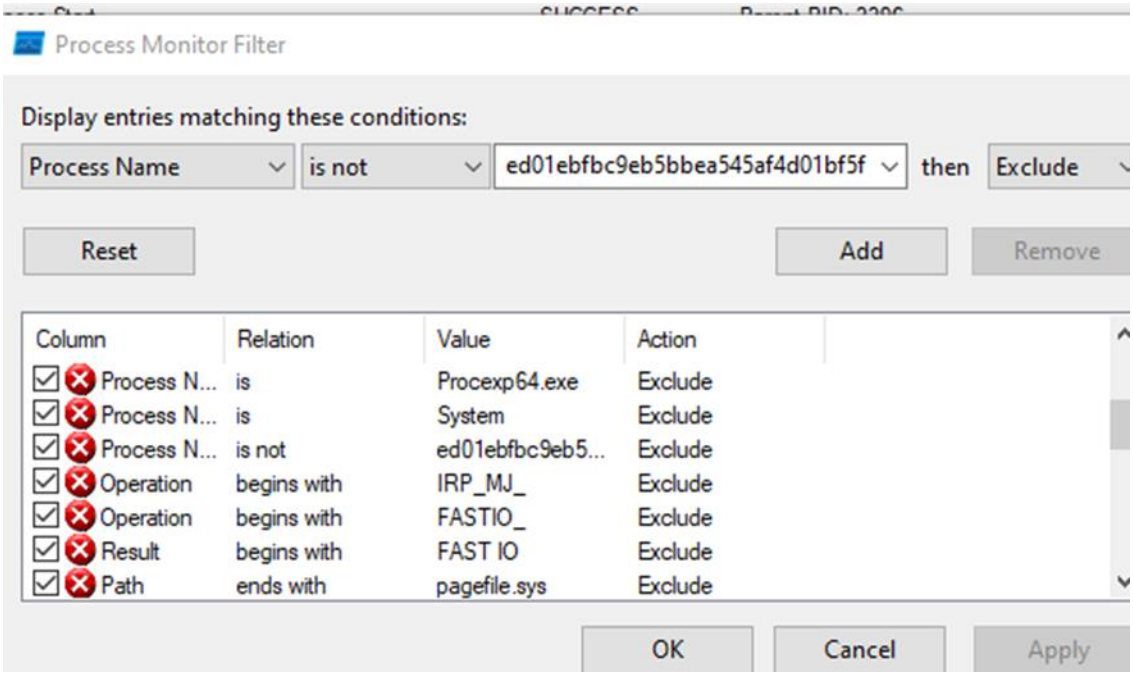


Figure 34: Process Monitor filters

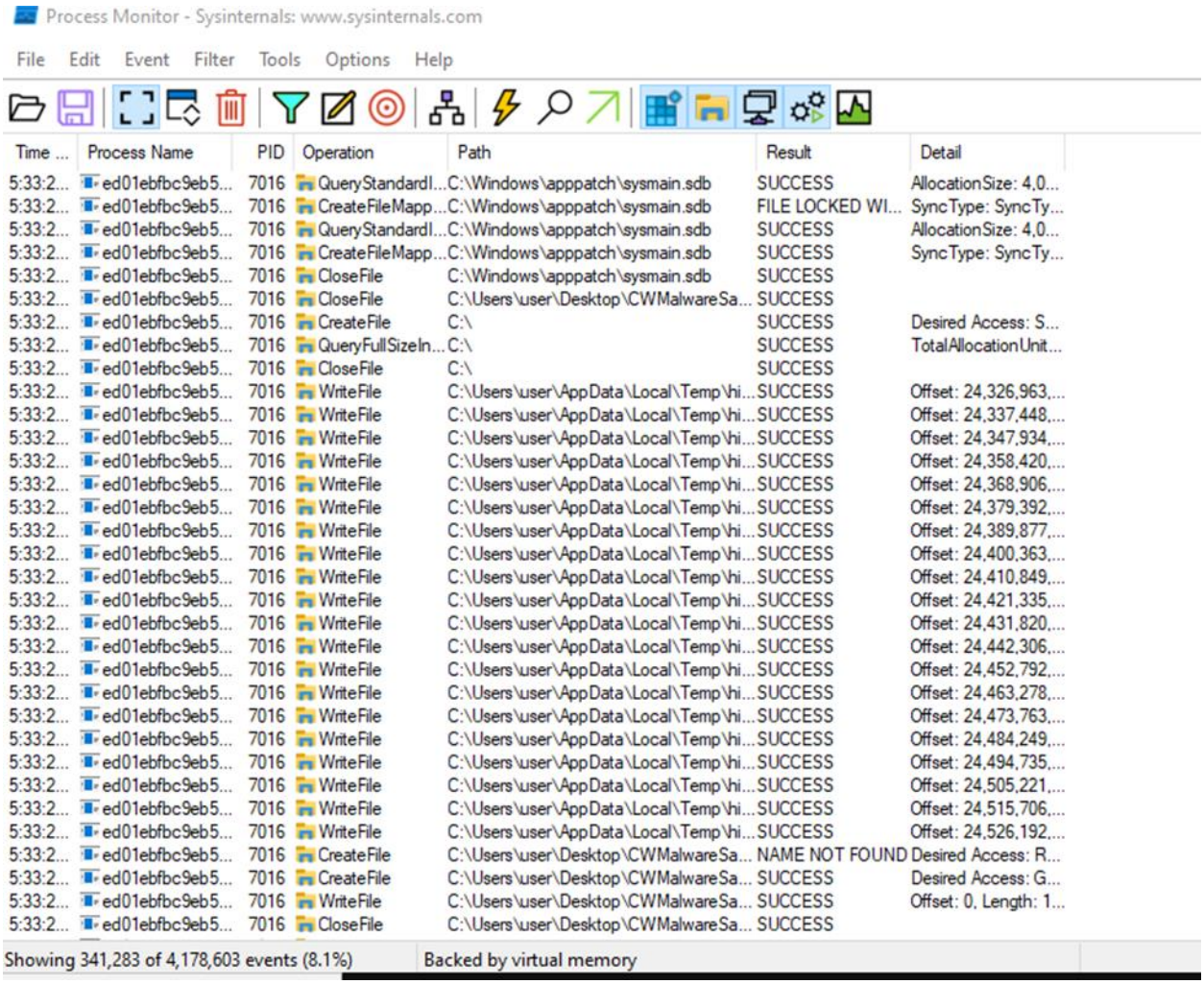


Figure 35: Process Monitor filtered

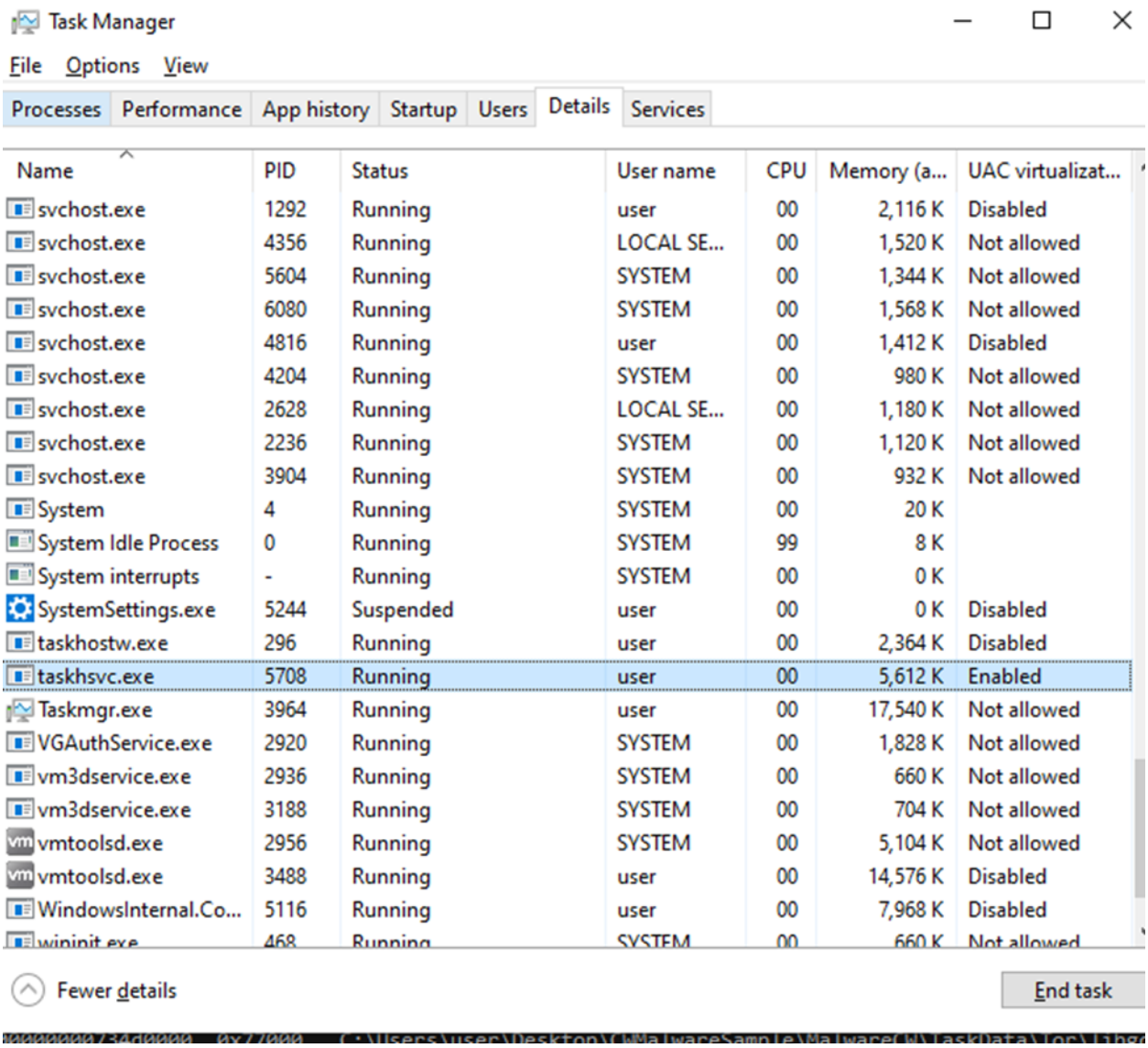


Figure 36: Task manager PID

```
C:\Windows\system32>listdlls 5708

Listdlls v3.2 - Listdlls
Copyright (C) 1997-2016 Mark Russinovich
Sysinternals

------------------------------------------------------------------------------
taskhsvc.exe pid: 5708
Command line: TaskData\Tor\taskhsvc.exe

Base                Size      Path
0x0000000000c70000  0x2fe000  C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\taskhsvc.exe
0x00000000fa360000  0x1f0000  C:\Windows\SYSTEM32\ntdll.dll
0x00000000fa0a0000  0x55000   C:\Windows\System32\wow64.dll
0x00000000f9780000  0x7d000   C:\Windows\System32\wow64win.dll
0x0000000077580000  0x9000    C:\Windows\System32\wow64cpu.dll
0x0000000000c70000  0x2fe000  C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\taskhsvc.exe
0x0000000077590000  0x19a000  C:\Windows\SysWOW64\ntdll.dll
0x0000000075ff0000  0xe0000   C:\Windows\SysWOW64\KERNEL32.DLL
0x0000000076c00000  0x1fc000  C:\Windows\SysWOW64\KERNELBASE.dll
0x0000000074840000  0x9f000   C:\Windows\SysWOW64\apphelp.dll
0x0000000075620000  0x79000   C:\Windows\SysWOW64\ADVAPI32.dll
0x0000000076b40000  0xbf000   C:\Windows\SysWOW64\msvcrt.dll
0x0000000075df0000  0x76000   C:\Windows\SysWOW64\sechost.dll
0x00000000774c0000  0xbb000   C:\Windows\SysWOW64\RPCRT4.dll
0x0000000074d60000  0x20000   C:\Windows\SysWOW64\SspiCli.dll
0x0000000074d50000  0xa000    C:\Windows\SysWOW64\CRYPTBASE.dll
0x00000000766a0000  0x5f000   C:\Windows\SysWOW64\bcryptPrimitives.dll
0x0000000076120000  0x57a000  C:\Windows\SysWOW64\SHELL32.dll
0x0000000075860000  0x11f000  C:\Windows\SysWOW64\ucrtbase.dll
0x00000000770c0000  0x3b000   C:\Windows\SysWOW64\cfgmgr32.dll
0x00000000756c0000  0x84000   C:\Windows\SysWOW64\shcore.dll
0x0000000076700000  0x276000  C:\Windows\SysWOW64\combase.dll
0x0000000074da0000  0x5c5000  C:\Windows\SysWOW64\windows.storage.dll
0x00000000773a0000  0x7c000   C:\Windows\SysWOW64\msvcp_win.dll
0x0000000077190000  0x17000   C:\Windows\SysWOW64\profapi.dll
0x0000000073550000  0x82000   C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\libevent-2-0-5.dll
0x0000000077350000  0x43000   C:\Windows\SysWOW64\powrprof.dll
0x0000000077100000  0x5e000   C:\Windows\SysWOW64\WS2_32.dll
0x00000000774b0000  0xd000    C:\Windows\SysWOW64\UMPDC.dll
0x00000000760d0000  0x44000   C:\Windows\SysWOW64\shlwapi.dll
0x0000000075390000  0x21000   C:\Windows\SysWOW64\GDI32.dll
0x0000000074d80000  0x17000   C:\Windows\SysWOW64\win32u.dll
0x00000000754c0000  0x15a000  C:\Windows\SysWOW64\gdi32full.dll
0x00000000771b0000  0x197000  C:\Windows\SysWOW64\USER32.dll
0x0000000075750000  0xf000    C:\Windows\SysWOW64\kernel.appcore.dll
0x00000000756a0000  0x13000   C:\Windows\SysWOW64\cryptsp.dll
0x00000000734d0000  0x77000   C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\libgcc_s_sjlj-1.dll
0x00000000734b0000  0x1c000   C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\libssp-0.dll
0x0000000073480000  0x22000   C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\zlib1.dll
0x00000000733f0000  0x82000   C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\SSLEAY32.dll
0x00000000731d0000  0x21c000  C:\Users\user\Desktop\CWMalwareSample\MalwareCW\TaskData\Tor\LIBEAY32.dll
0x0000000073cb0000  0x2f000   C:\Windows\SysWOW64\rsaenh.dll
0x0000000077160000  0x19000   C:\Windows\SysWOW64\bcrypt.dll
```

Figure 37: Strings using listdlls

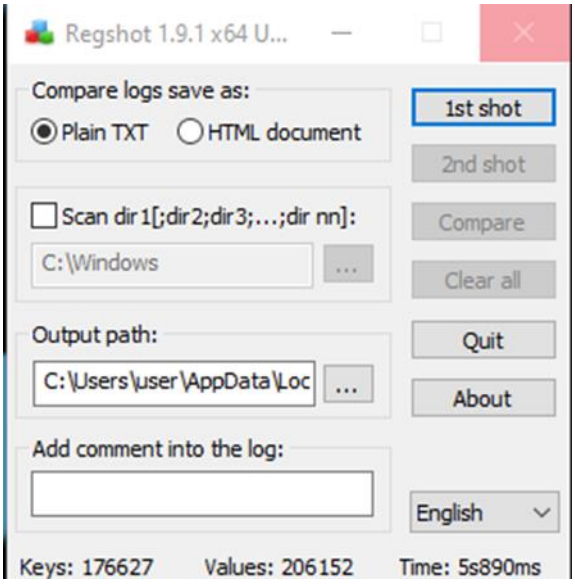


Figure 38: Regshot snapshots

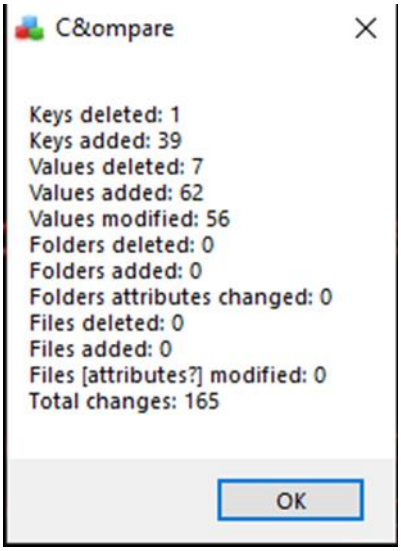


Figure 39: Regshot snapshot comparison

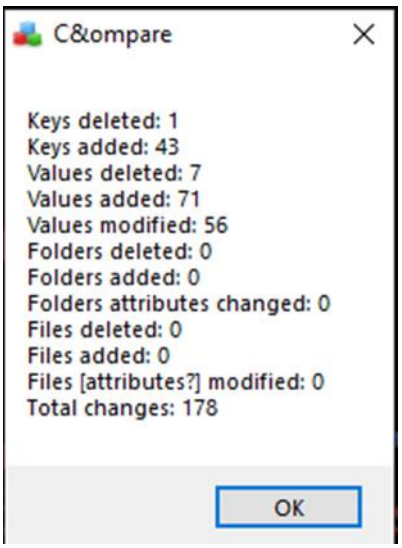


Figure 40: Regshot snapshot comparison 2

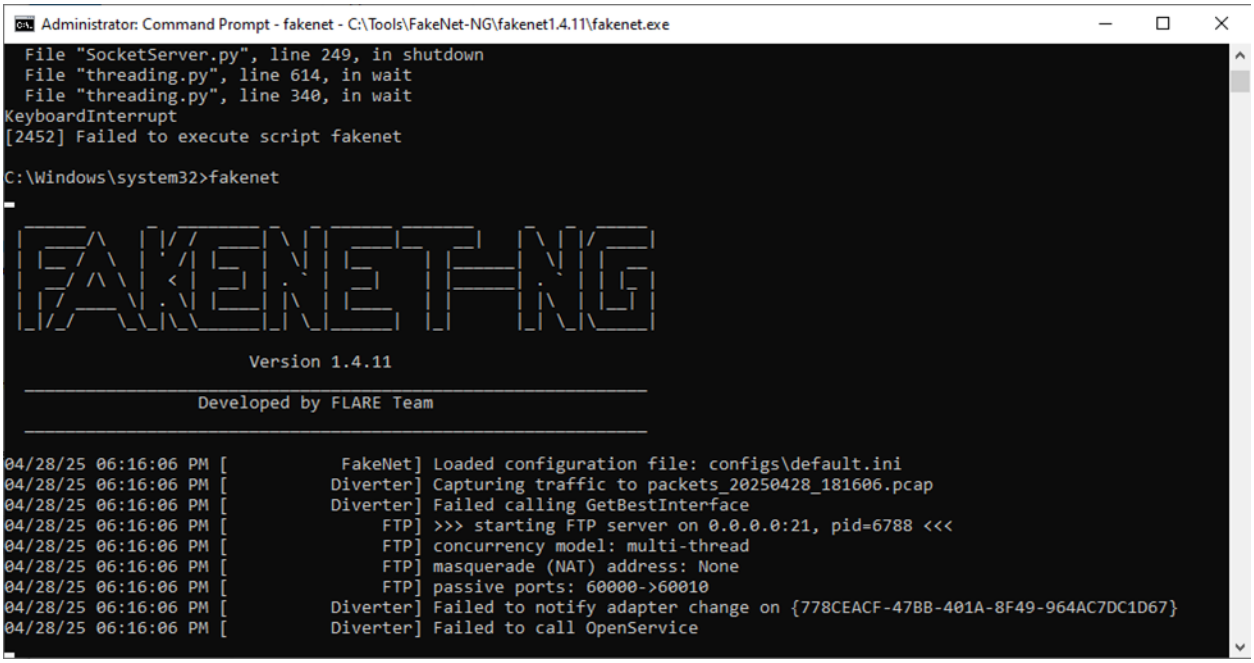


Figure 41: FakeNet-NG setup

```
04/28/25 06:21:26 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:21:29 PM [          Diverter] @WanaDecryptor@.exe (2244) requested TCP 127.0.0.1:9050
04/28/25 06:21:29 PM [          Diverter] taskhsvc.exe (5532) requested TCP 127.0.0.1:49700
04/28/25 06:21:29 PM [          Diverter] @WanaDecryptor@.exe (2244) requested TCP 127.0.0.1:9050
04/28/25 06:21:29 PM [          Diverter] taskhsvc.exe (5532) requested TCP 127.0.0.1:49700
04/28/25 06:21:29 PM [          Diverter] taskhsvc.exe (5532) requested TCP 127.0.0.1:49701
04/28/25 06:21:29 PM [          Diverter] @WanaDecryptor@.exe (2244) requested TCP 127.0.0.1:9050
04/28/25 06:21:29 PM [          Diverter] taskhsvc.exe (5532) requested TCP 127.0.0.1:49701
04/28/25 06:21:29 PM [          Diverter] @WanaDecryptor@.exe (2244) requested TCP 127.0.0.1:9050
04/28/25 06:21:29 PM [          Diverter] taskhsvc.exe (5532) requested TCP 127.0.0.1:49701
04/28/25 06:21:34 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:21:41 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:21:44 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:21:45 PM [          Diverter] System (4) requested UDP 192.168.20.255:137
04/28/25 06:21:48 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:21:51 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:21:54 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:21:58 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
04/28/25 06:22:06 PM [          Diverter] ICMP type 3 code 1 192.168.20.10->192.168.20.10
```

Figure 42: FakeNet-NG traffic

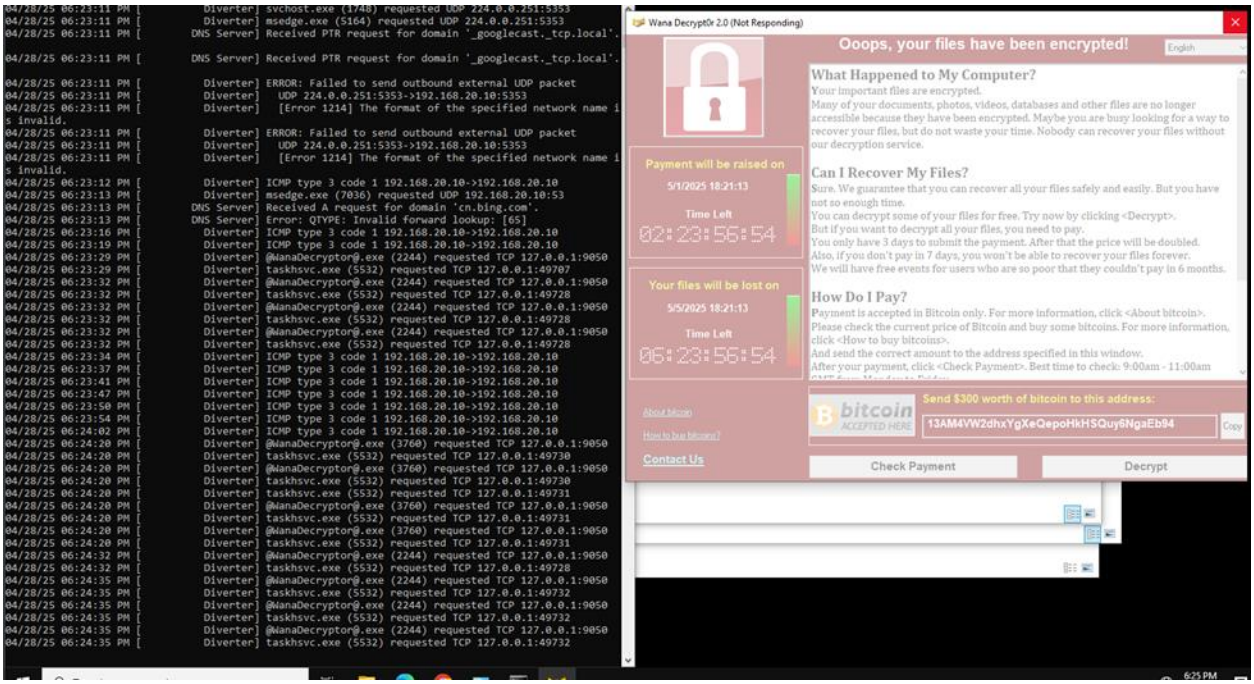


Figure 43: FakeNet-NG traffic 2

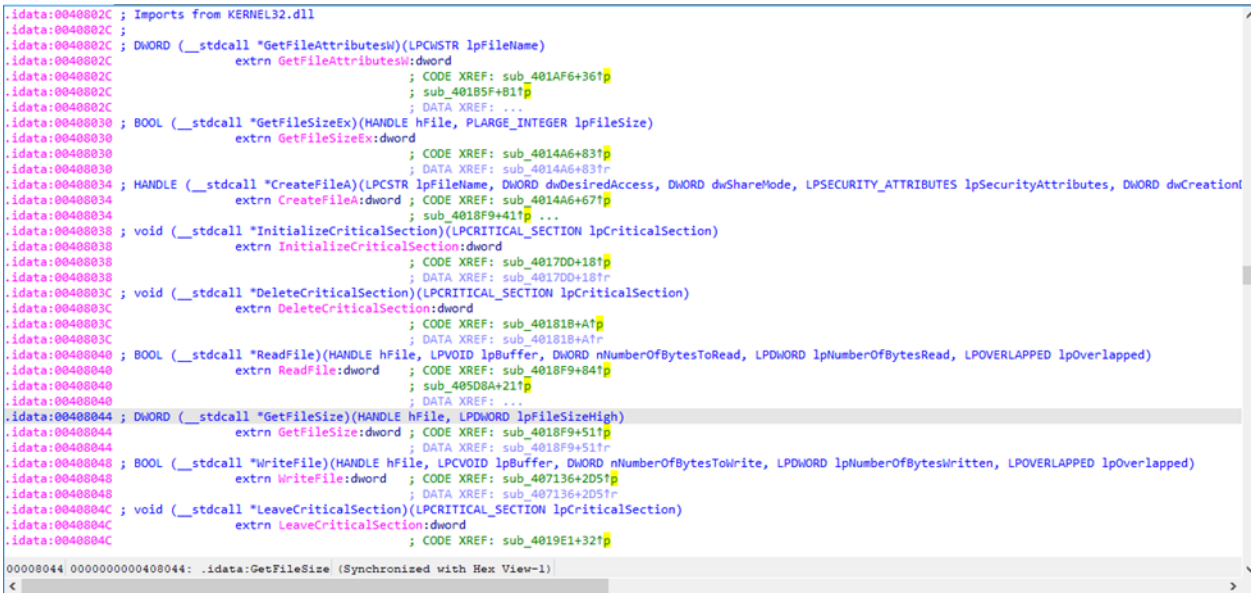


Figure 44: IDA KERNEL43.dll

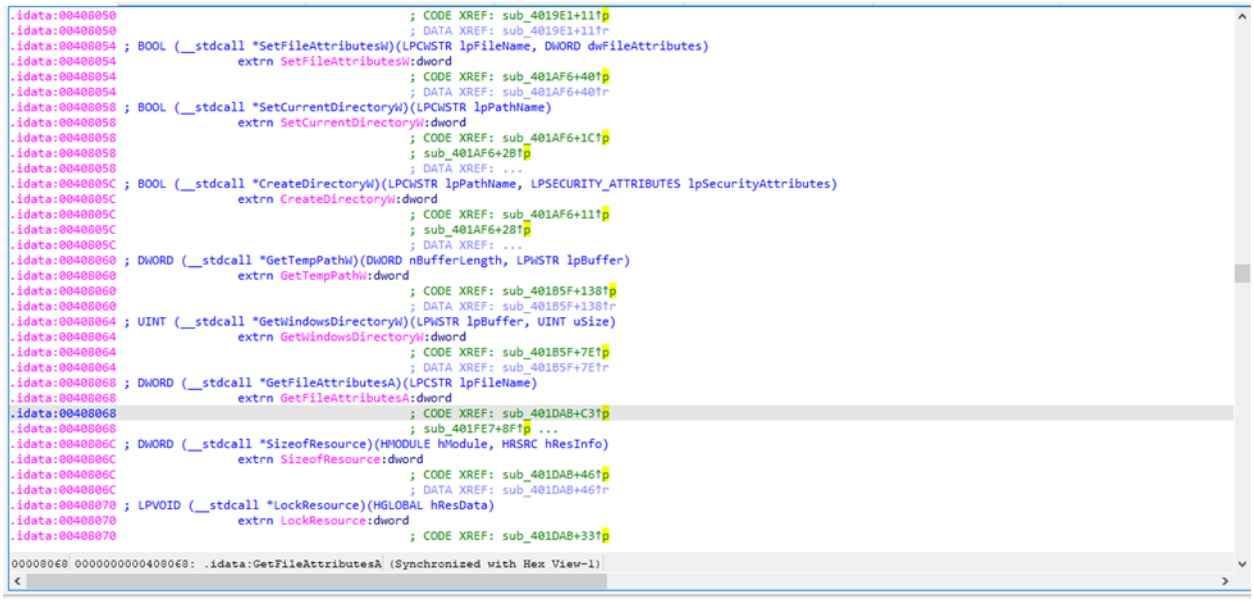


Figure 45: IDA KERNEL.dll 2

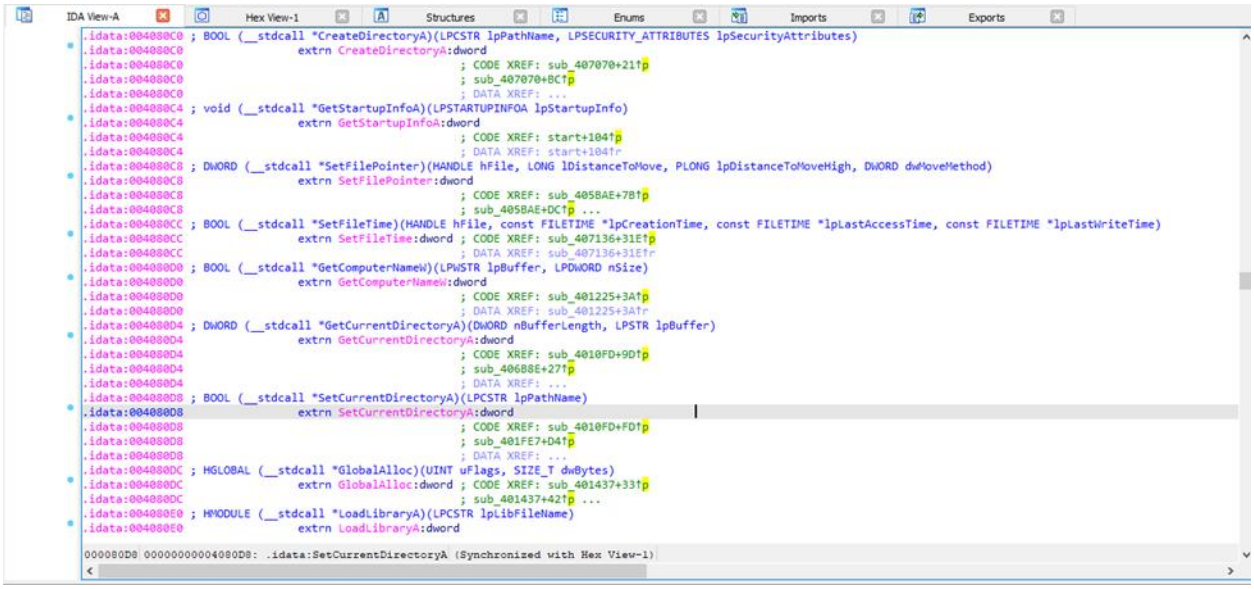


Figure 46: IDA Startup/file modification

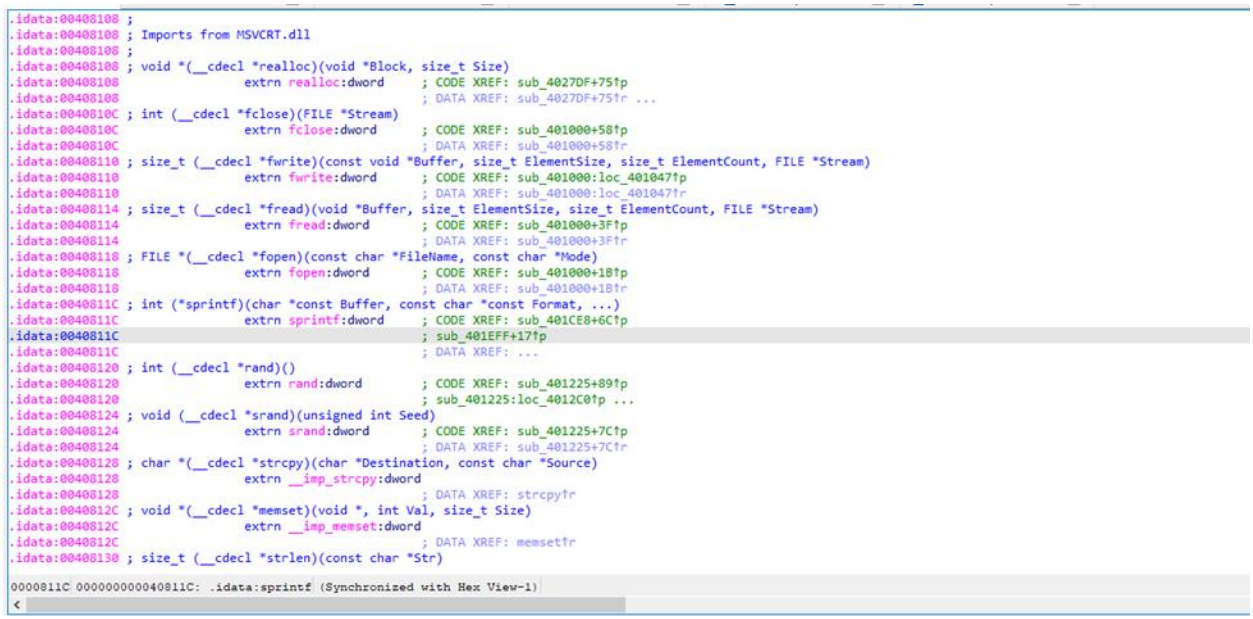


Figure 47: IDA MSVCRT.dll import

```
.idata:004081D0 ; Imports from USER32.dll
.idata:004081D0 ;
.idata:004081D0 ; int (*wsprintfA)(LPSTR, LPCSTR, ...)
.idata:004081D0                 extrn wsprintfA:dword   ; CODE XREF: sub_407136+1C6↑p
.idata:004081D0                                         ; sub_407136+259↑p
.idata:004081D0                                         ; DATA XREF: ...
.idata:004081D4
.idata:004081D4
```

Figure 48: IDA USER32.dll import

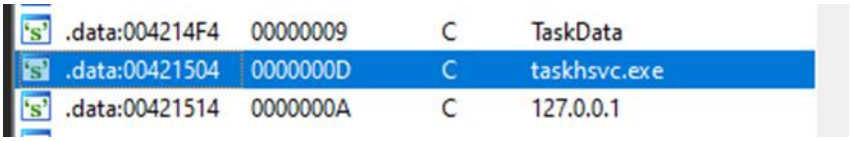
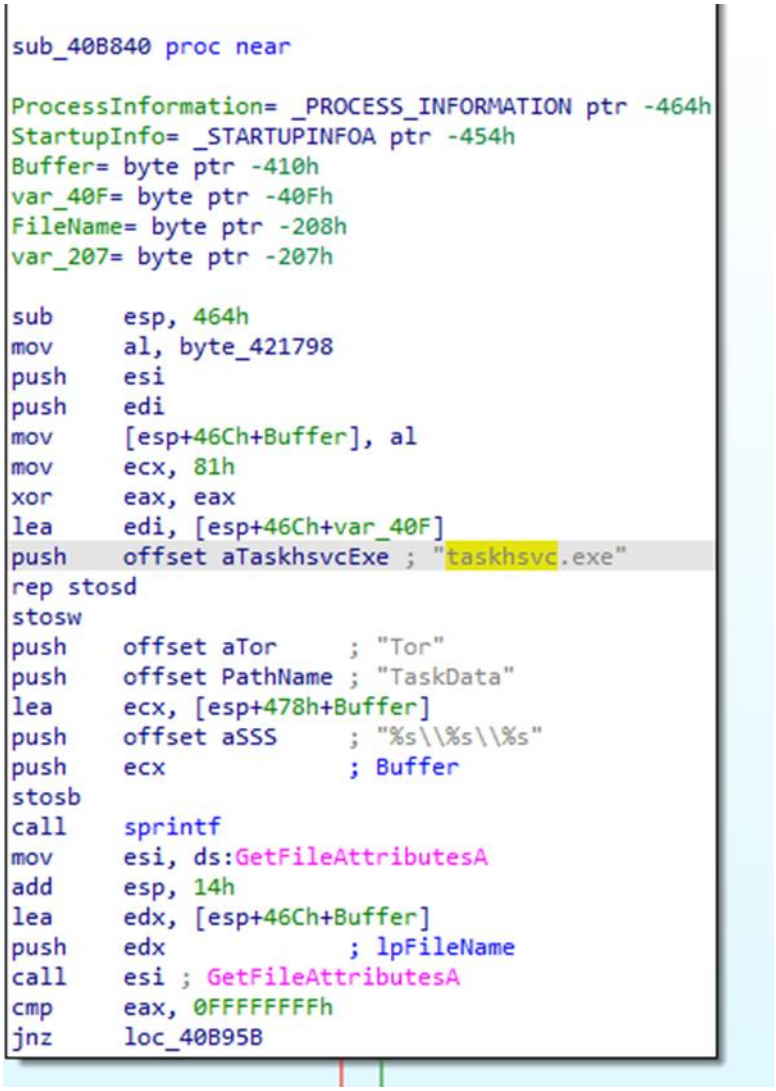


Figure 49: IDA search "taskhsvc.exe"

```
.data:004214F4 ; CHAR PathName[]
.data:004214F4 PathName        db 'TaskData',0          ; DATA XREF: sub_40B840+2A↑o
.data:004214F4                                          ; sub_40B840+5F↑o ...
.data:004214FD                 align 10h
.data:00421500 aTor            db 'Tor',0               ; DATA XREF: sub_40B840+25↑o
.data:00421500                                          ; sub_40B840+CC↑o
.data:00421504 aTaskhsvcExe    db 'taskhsvc.exe',0      ; DATA XREF: sub_40B840+1C↑o
.data:00421511                 align 4
.data:00421514 a127001         db '127.0.0.1',0         ; DATA XREF: sub_40BA10+D↑o
.data:00421514                                          ; sub_40BA10+36↑o ...
.data:0042151E                 align 10h
```

Figure 50: IDA search result



```
sub_40B840 proc near

ProcessInformation= _PROCESS_INFORMATION ptr -464h
StartupInfo= _STARTUPINFOA ptr -454h
Buffer= byte ptr -410h
var_40F= byte ptr -40Fh
FileName= byte ptr -208h
var_207= byte ptr -207h

sub     esp, 464h
mov     al, byte_421798
push    esi
push    edi
mov     [esp+46Ch+Buffer], al
mov     ecx, 81h
xor     eax, eax
lea     edi, [esp+46Ch+var_40F]
push    offset aTaskhsvcExe ; "taskhsvc.exe"
rep stosd
stosw
push    offset aTor     ; "Tor"
push    offset PathName ; "TaskData"
lea     ecx, [esp+478h+Buffer]
push    offset aSSS     ; "%s\\%s\\%s"
push    ecx             ; Buffer
stosb
call    sprintf
mov     esi, ds:GetFileAttributesA
add     esp, 14h
lea     edx, [esp+46Ch+Buffer]
push    edx             ; lpFileName
call    esi ; GetFileAttributesA
cmp     eax, 0FFFFFFFFh
jnz     loc_40B95B
```

Figure 51: IDA taskhsvc.exe launched by parent process

```
.data:0040F080                 db  93h ; “
.data:0040F081                 db 0C1h ; Á
.data:0040F082                 db    0
.data:0040F083                 db  0Eh
.data:0040F084                 db 0F1h ; ñ
.data:0040F085                 db 0A9h ; ©
.data:0040F086                 db  25h ; %
.data:0040F087                 db 0C8h ; È
.data:0040F088                 db 0F6h ; ö
.data:0040F089                 db 0E8h ; è
.data:0040F08A                 db  8Bh ; ‹
.data:0040F08B                 db 0C7h ; Ç
.data:0040F08C aMicrosoftEnhan db 'Microsoft Enhanced RSA and AES Cryptographic Provider',0
.data:0040F08C                                         ; DATA XREF: sub_40182C+14↑o
.data:0040F0C2                 align 4
.data:0040F0C4 ; CHAR aCryptgenkey[]
.data:0040F0C4 aCryptgenkey    db 'CryptGenKey',0      ; DATA XREF: sub_401A45+68↑o
.data:0040F0D0 ; CHAR aCryptdecrypt[]
.data:0040F0D0 aCryptdecrypt   db 'CryptDecrypt',0     ; DATA XREF: sub_401A45+5B↑o
.data:0040F0DD                 align 10h
.data:0040F0E0 ; CHAR aCryptencrypt[]
.data:0040F0E0 aCryptencrypt   db 'CryptEncrypt',0     ; DATA XREF: sub_401A45+4E↑o
.data:0040F0ED                 align 10h
.data:0040F0F0 ; CHAR aCryptdestroyke[]
.data:0040F0F0 aCryptdestroyke db 'CryptDestroyKey',0  ; DATA XREF: sub_401A45+41↑o
.data:0040F100 ; CHAR aCryptimportkey[]
.data:0040F100 aCryptimportkey db 'CryptImportKey',0   ; DATA XREF: sub_401A45+34↑o
.data:0040F10F                 align 10h
.data:0040F110 ; CHAR aCryptacquireco[]
.data:0040F110 aCryptacquireco db 'CryptAcquireContextA',0
.data:0040F110                                         ; DATA XREF: sub_401A45+2C↑o
.data:0040F125                 align 4
.data:0040F128                 dd offset aDoc          ; ".doc"
.data:0040F12C                 dd offset aDocx         ; ".docx"
.data:0040F130                 dd offset aDocb         ; ".docb"
.data:0040F134                 dd offset aDocm         ; ".docm"
```

Figure 52: IDA cryptography functions

```
00401173  .∨EB 05          JMP SHORT ed01ebfb.0040117A
00401175  > 68 01000080    PUSH 80000001                    hKey = HKEY_CURRENT_USER
0040117A  > FF15 14804000  CALL DWORD PTR DS:[<&ADVAPI32.RegCreat  RegCreateKeyW
00401180  . 3975 FC        CMP DWORD PTR SS:[EBP-4],ESI
00401183  .∨0F84 84000000  JE ed01ebfb.0040120D
00401189  . 3975 08        CMP DWORD PTR SS:[EBP+8],ESI
0040118C  .∨74 3E          JE SHORT ed01ebfb.004011CC
0040118E  . 8D85 24FDFFFF  LEA EAX,DWORD PTR SS:[EBP-2DC]
00401194  . 50             PUSH EAX                         Buffer
00401195  . 68 07020000    PUSH 207                         BufSize = 207 (519.)
0040119A  . FF15 D4804000  CALL DWORD PTR DS:[<&KERNEL32.GetCurrer  GetCurrentDirectoryA
004011A0  . 8D85 24FDFFFF  LEA EAX,DWORD PTR SS:[EBP-2DC]
004011A6  . 50             PUSH EAX                         s
004011A7  . E8 08650000    CALL <JMP.&MSVCRT.strlen>        strlen
004011AC  . 59             POP ECX
004011AD  . 40             INC EAX
004011AE  . 50             PUSH EAX                         BufSize
004011AF  . 8D85 24FDFFFF  LEA EAX,DWORD PTR SS:[EBP-2DC]
004011B5  . 50             PUSH EAX                         Buffer
004011B6  . 6A 01          PUSH 1                           ValueType = REG_SZ
004011B8  . 56             PUSH ESI                         Reserved
004011B9  . 57             PUSH EDI                         ValueName
004011BA  . FF75 FC        PUSH DWORD PTR SS:[EBP-4]        hKey
004011BD  . FF15 18804000  CALL DWORD PTR DS:[<&ADVAPI32.RegSetVa  RegSetValueExA
004011C3  . 8BF0           MOV ESI,EAX
004011C5  . F7DE           NEG ESI
004011C7  . 1BF6           SBB ESI,ESI
004011C9  . 46             INC ESI
004011CA  .∨EB 34          JMP SHORT ed01ebfb.00401200
004011CC  > 8D45 F4        LEA EAX,DWORD PTR SS:[EBP-C]
004011CF  . C745 F4 07020(  MOV DWORD PTR SS:[EBP-C],207
```

Figure 53: Ollydbg 1

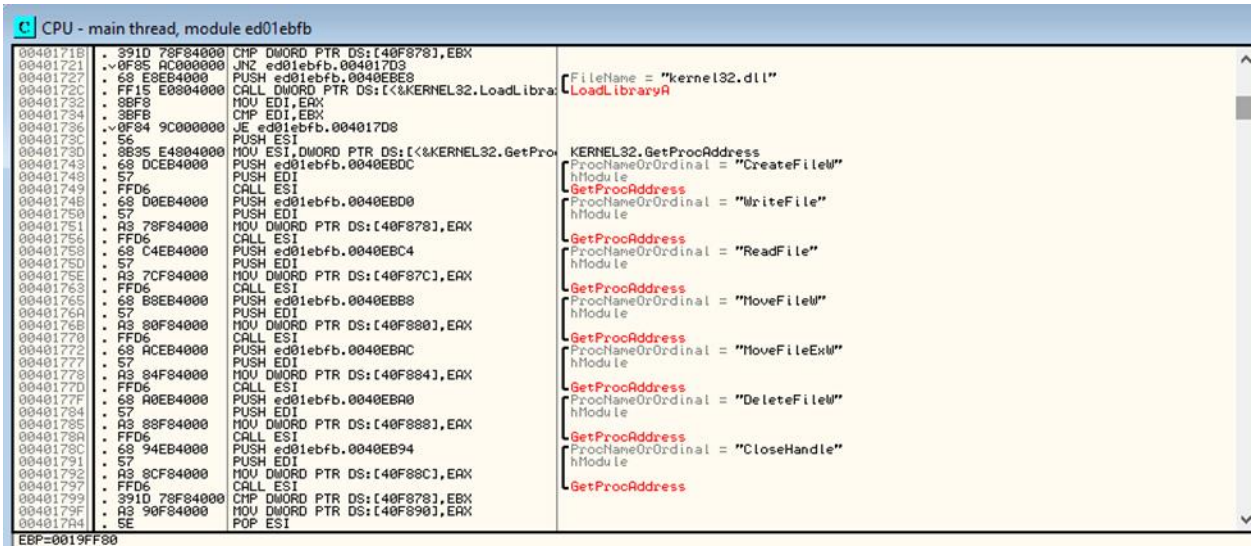


Figure 54: Ollydbg 2

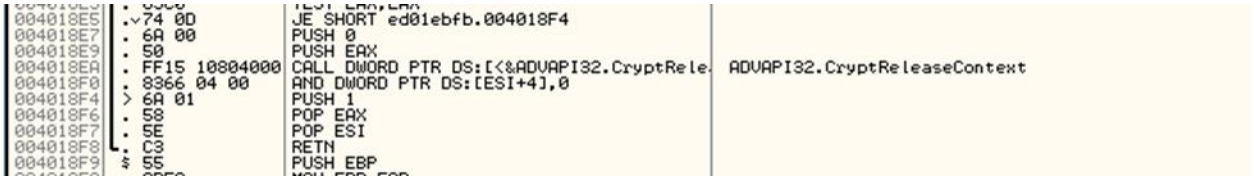


Figure 55: Ollydbg 3

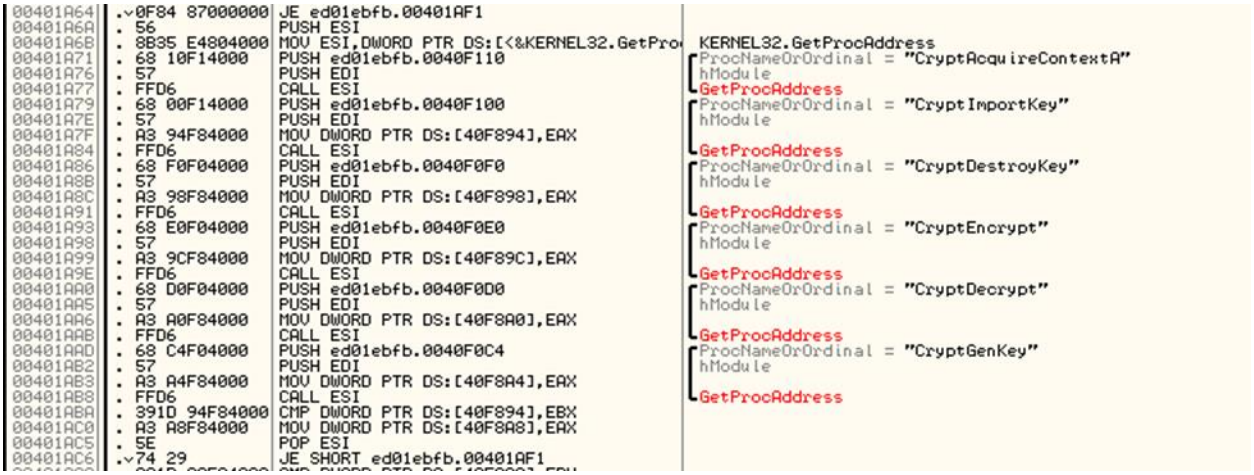
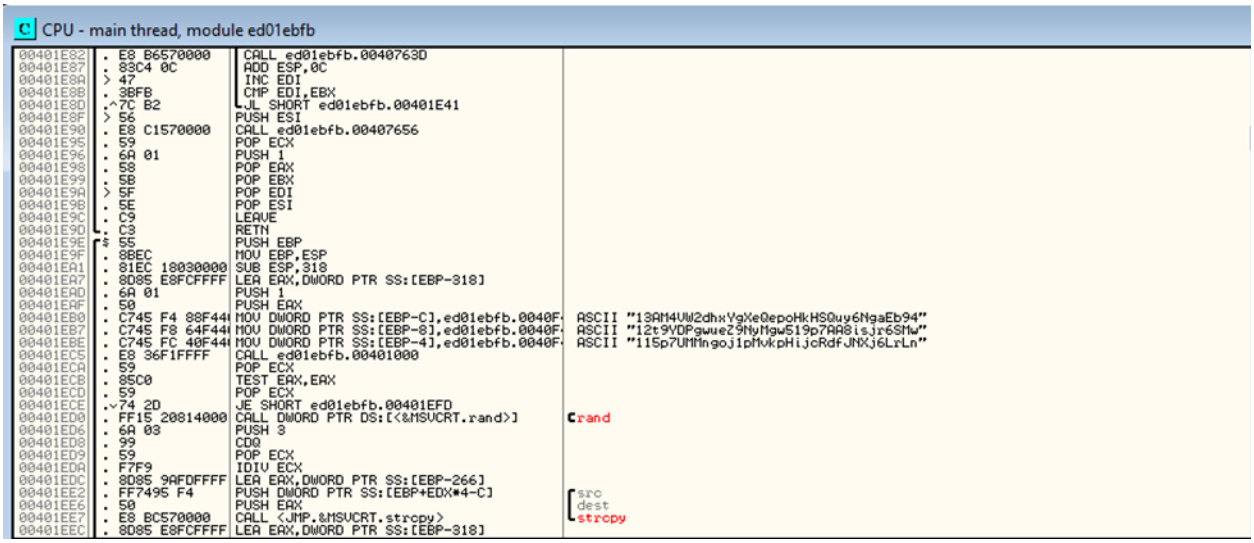


Figure 56: Ollydbg 4

```
00401AF7  . 8BEC          MOV EBP,ESP
00401AF9  . 53            PUSH EBX
00401AFA  . 56            PUSH ESI
00401AFB  . 8B35 5C804000 MOV ESI,DWORD PTR DS:[<&KERNEL32.CreateI  KERNEL32.CreateDirectoryW
00401B01  . 57            PUSH EDI
00401B02  . 6A 00         PUSH 0                                    pSecurity = NULL
00401B04  . FF75 08       PUSH DWORD PTR SS:[EBP+8]                 Path
00401B07  . FFD6          CALL ESI                                  CreateDirectoryW
00401B09  . FF75 08       PUSH DWORD PTR SS:[EBP+8]                 Path
00401B0C  . 8B3D 58804000 MOV EDI,DWORD PTR DS:[<&KERNEL32.SetCur:  KERNEL32.SetCurrentDirectoryW
00401B12  . FFD7          CALL EDI                                  SetCurrentDirectoryW
00401B14  . 85C0          TEST EAX,EAX
00401B16  .v74 0F         JE SHORT ed01ebfb.00401B27
00401B18  . 8B5D 0C       MOV EBX,DWORD PTR SS:[EBP+C]
00401B1B  . 6A 00         PUSH 0                                    pSecurity = NULL
00401B1D  . 53            PUSH EBX                                  Path
00401B1E  . FFD6          CALL ESI                                  CreateDirectoryW
00401B20  . 53            PUSH EBX                                  Path
00401B21  . FFD7          CALL EDI                                  SetCurrentDirectoryW
00401B23  . 85C0          TEST EAX,EAX
00401B25  .v75 04         JNZ SHORT ed01ebfb.00401B2B
00401B27  > 33C0          XOR EAX,EAX
00401B29  .vEB 2F         JMP SHORT ed01ebfb.00401B5A
00401B2B  > 53            PUSH EBX                                  FileName
00401B2C  . FF15 2C804000 CALL DWORD PTR DS:[<&KERNEL32.GetFileAt   GetFileAttributesW
00401B32  . 0C 06         OR AL,6
00401B34  . 50            PUSH EAX                                  FileAttributes
00401B35  . 53            PUSH EBX                                  FileName
00401B36  . FF15 54804000 CALL DWORD PTR DS:[<&KERNEL32.SetFileAt   SetFileAttributesW
00401B3C  . 837D 10 00    CMP DWORD PTR SS:[EBP+10],0
00401B40  .v74 15         JE SHORT ed01ebfb.00401B57
00401B42  . 53            PUSH EBX                                  <%s>
00401B43  . FF75 08       PUSH DWORD PTR SS:[EBP+8]                 <%s>
00401B46  . 68 88EB4000   PUSH ed01ebfb.0040EB88                    format = "%s\%s"
00401B4B  . FF75 10       PUSH DWORD PTR SS:[EBP+10]                wstr
00401B4E  . FF15 54814000 CALL DWORD PTR DS:[<&MSVCRT.swprintf>]    swprintf
00401B54  . 83C4 10       ADD ESP,10
00401B57  > 6A 01         PUSH 1
```

Figure 57: Ollydbg 5



CPU - main thread, module ed01ebfb

```
00401E82  . E8 B6570000   CALL ed01ebfb.0040763D
00401E87  . 83C4 0C       ADD ESP,0C
00401E8A  > 47            INC EDI
00401E8B  . 3BFB          CMP EDI,EBX
00401E8D  .^7C B2         JL SHORT ed01ebfb.00401E41
00401E8F  > 56            PUSH ESI
00401E90  . E8 C1570000   CALL ed01ebfb.00407656
00401E95  . 59            POP ECX
00401E96  . 6A 01         PUSH 1
00401E98  . 58            POP EAX
00401E99  . 5B            POP EBX
00401E9A  > 5F            POP EDI
00401E9B  . 5E            POP ESI
00401E9C  . C9            LEAVE
00401E9D  . C3            RETN
00401E9E  $ 55            PUSH EBP
00401E9F  . 8BEC          MOV EBP,ESP
00401EA1  . 81EC 18030000 SUB ESP,318
00401EA7  . 8D85 E8FCFFFF LEA EAX,DWORD PTR SS:[EBP-318]
00401EAD  . 6A 01         PUSH 1
00401EAF  . 50            PUSH EAX
00401EB0  . C745 F4 88F44( MOV DWORD PTR SS:[EBP-C],ed01ebfb.0040F   ASCII "13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb94"
00401EB7  . C745 F8 64F44( MOV DWORD PTR SS:[EBP-8],ed01ebfb.0040F   ASCII "12t9VDPgwueZ9NyMgw519p7AA8isjr6SMw"
00401EBE  . C745 FC 40F44( MOV DWORD PTR SS:[EBP-4],ed01ebfb.0040F   ASCII "115p7UMMngoj1pMvkpHijcRdfJNXj6LrLn"
00401EC5  . E8 36F1FFFF   CALL ed01ebfb.00401000
00401ECA  . 59            POP ECX
00401ECB  . 85C0          TEST EAX,EAX
00401ECD  . 59            POP ECX
00401ECE  .v74 2D         JE SHORT ed01ebfb.00401EFD
00401ED0  . FF15 20814000 CALL DWORD PTR DS:[<&MSVCRT.rand>]         rand
00401ED6  . 6A 03         PUSH 3
00401ED8  . 99            CDQ
00401ED9  . 59            POP ECX
00401EDA  . F7F9          IDIV ECX
00401EDC  . 8D85 9AFDFFFF LEA EAX,DWORD PTR SS:[EBP-266]
00401EE2  . FF7495 F4     PUSH DWORD PTR SS:[EBP+EDX*4-C]           src
00401EE6  . 50            PUSH EAX                                  dest
00401EE7  . E8 BC570000   CALL <JMP.&MSVCRT.strcpy>                 strcpy
00401EEC  . 8D85 E8FCFFFF LEA EAX,DWORD PTR SS:[EBP-318]
```

Figure 58: Ollydbg hardcoded wallets

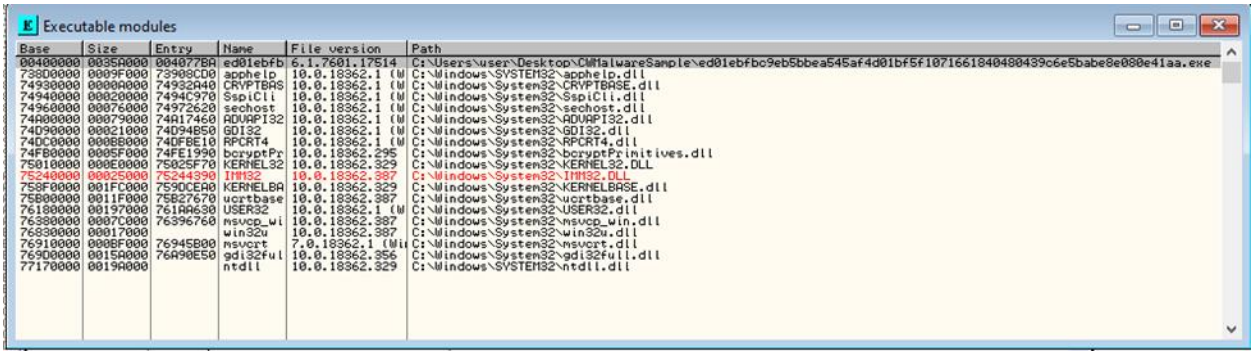


Figure 59: Ollydbg executable modules tab

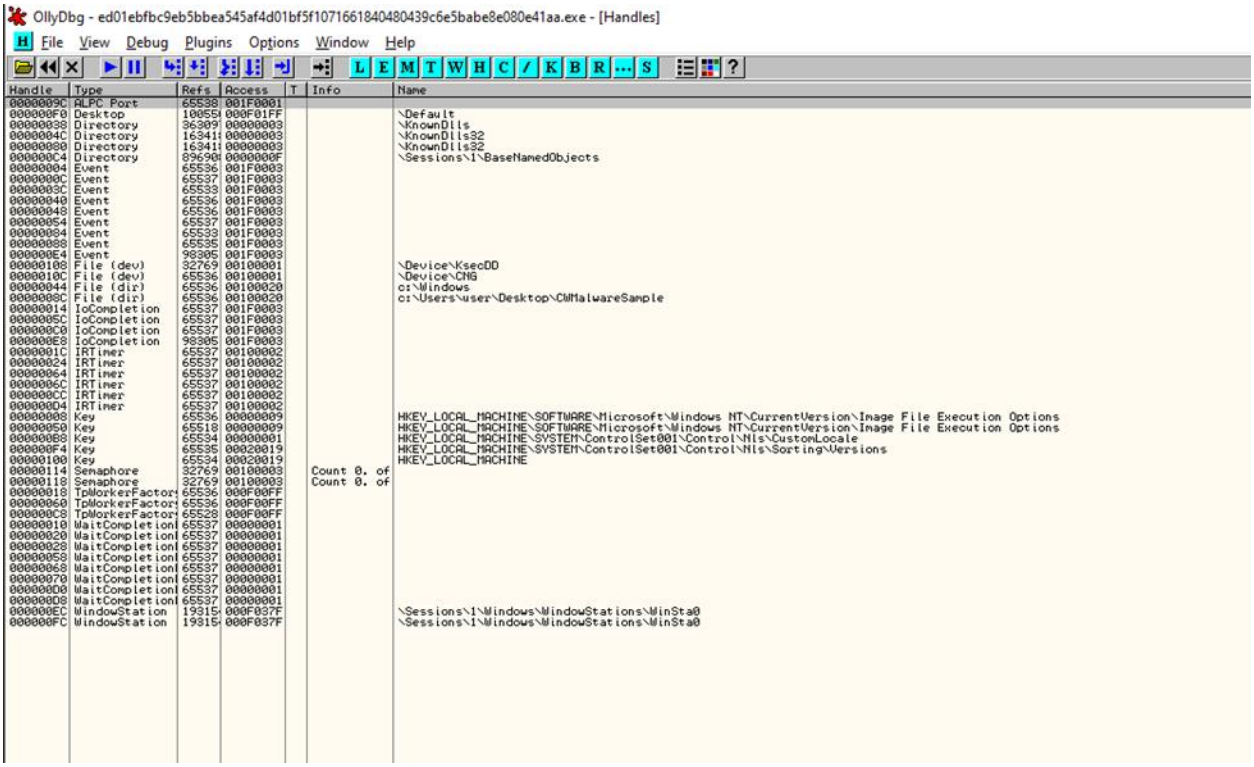


Figure 60: Ollydbg registry keys/directories

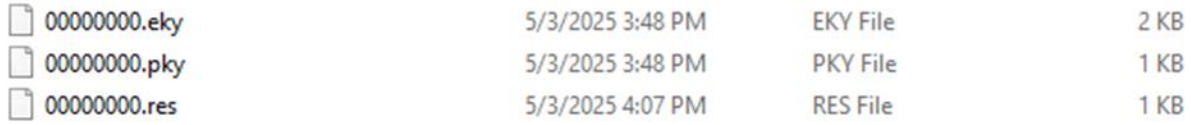


Figure 61: File Explorer RSA encryption related files

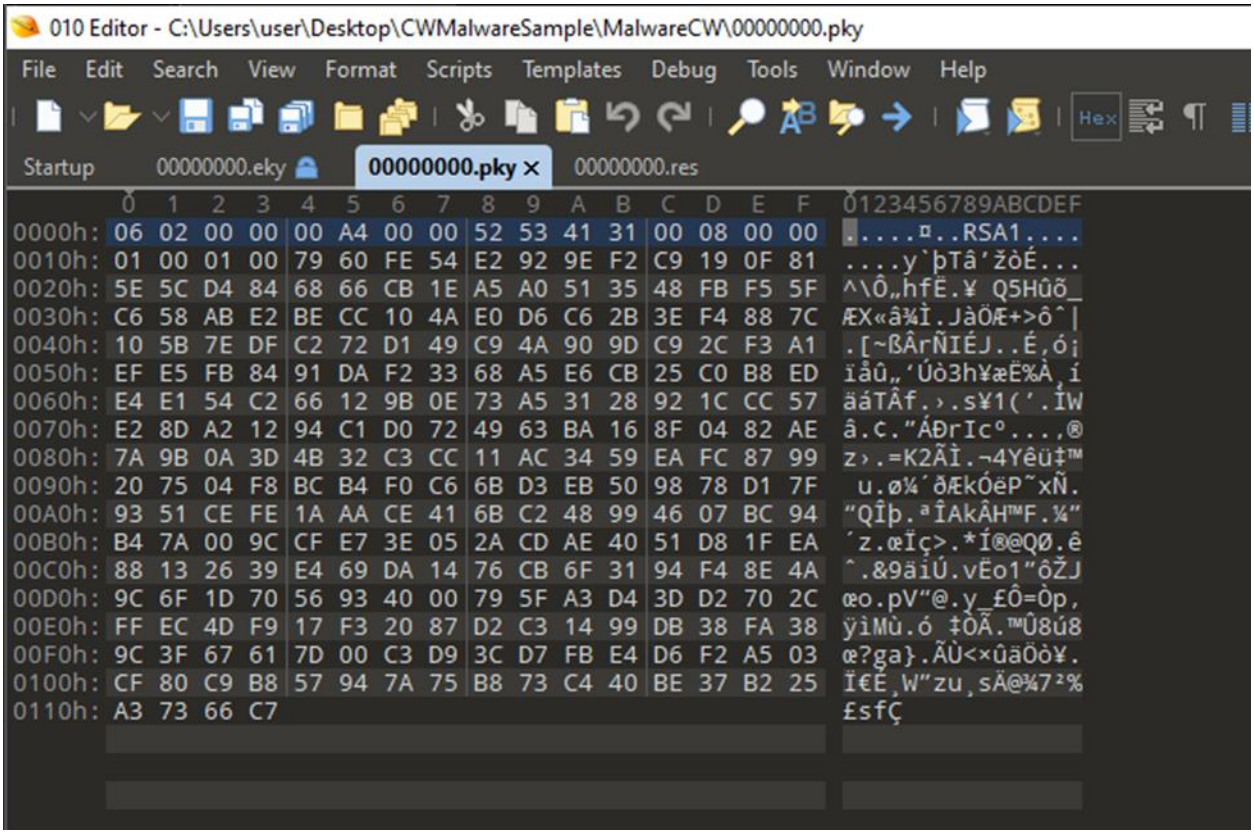


Figure 62: 010 Editor 00000000.pky

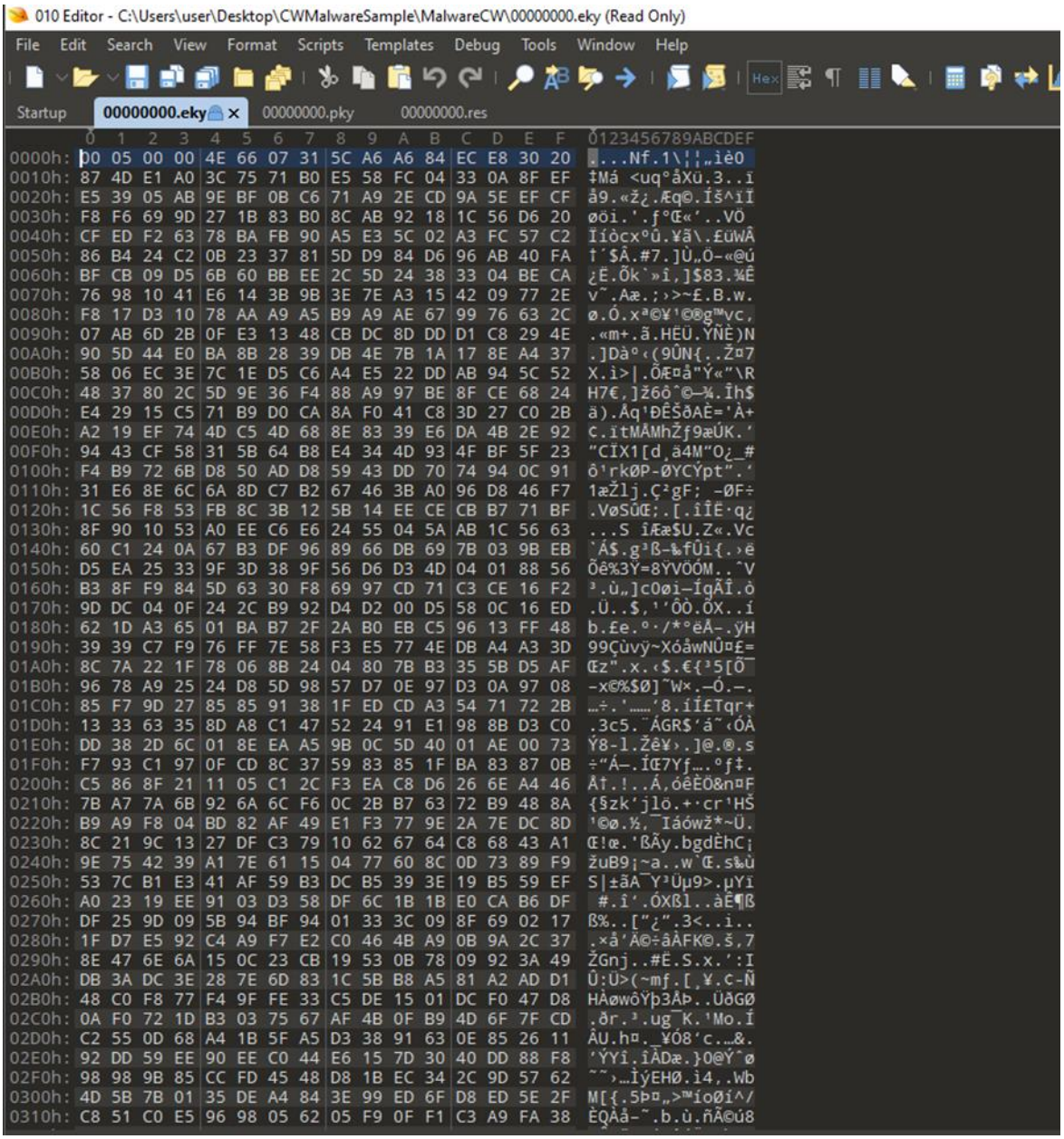


Figure 63: 010 Editor 00000000.eky

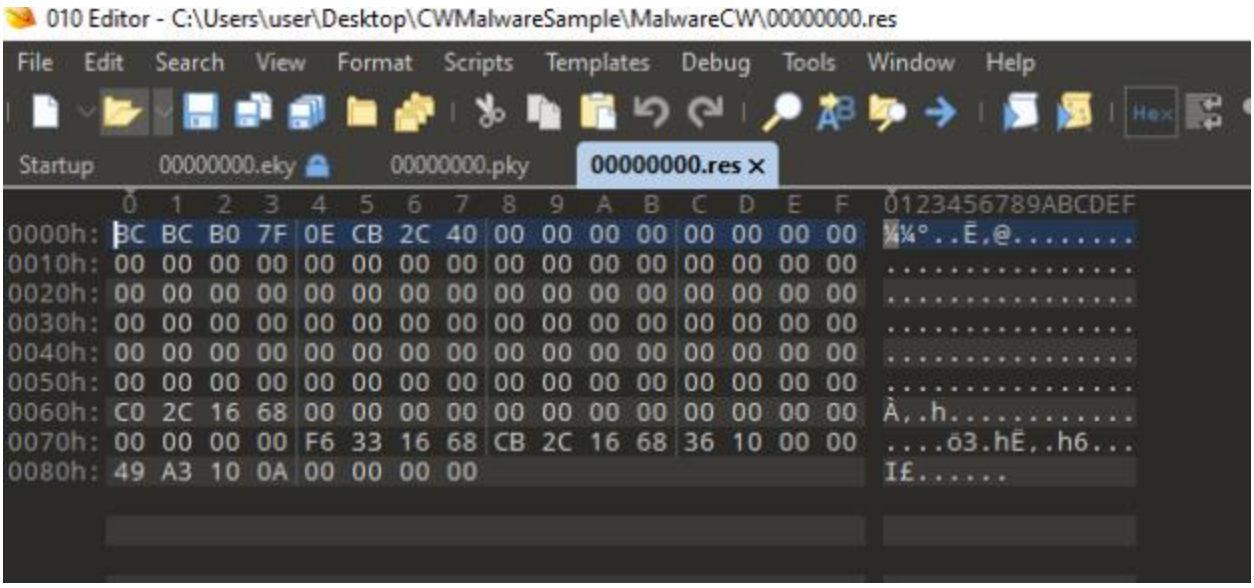


Figure 64: 010 Editor 00000000.res

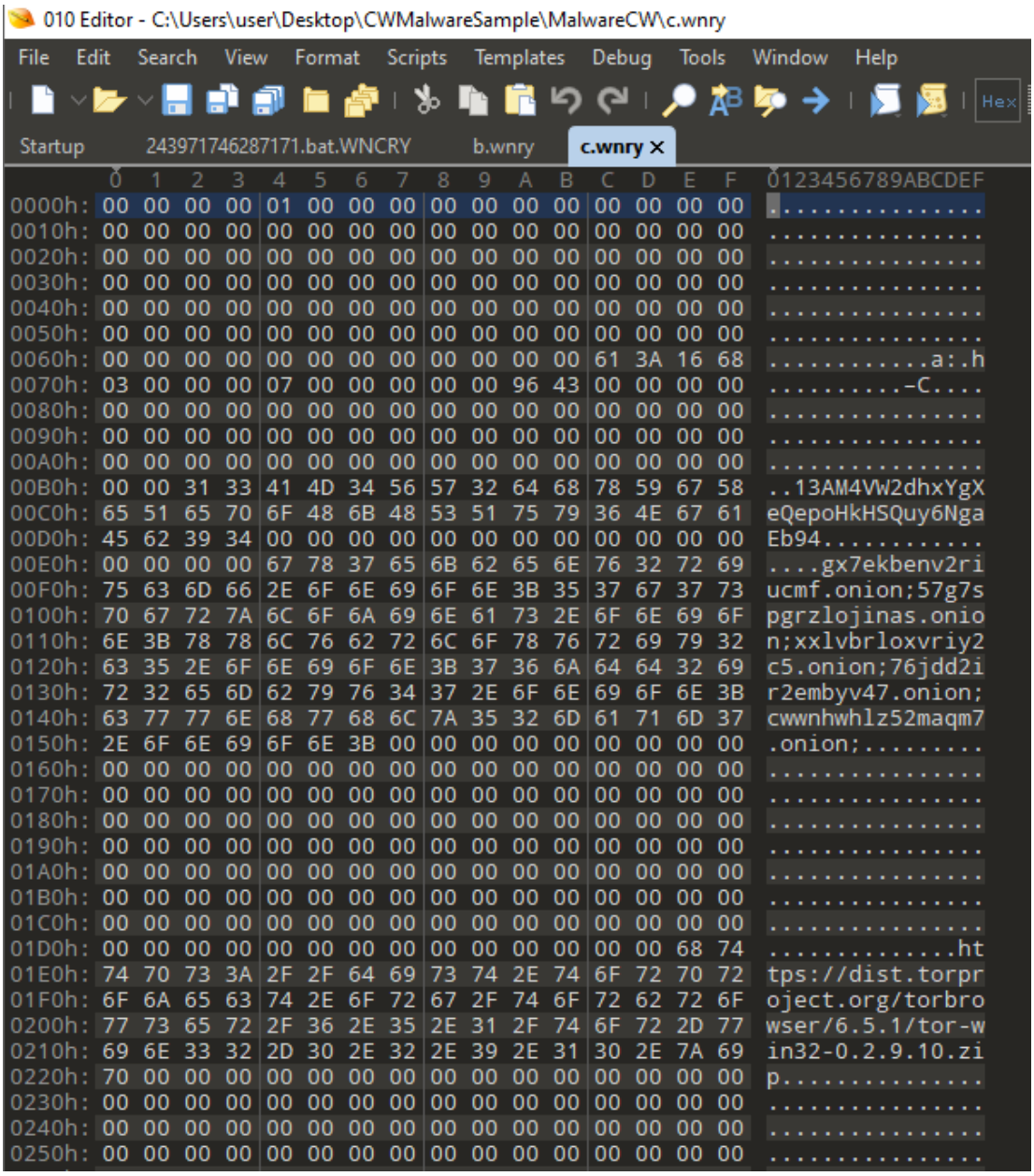


Figure 65: 010 Editor "c.wnry" file

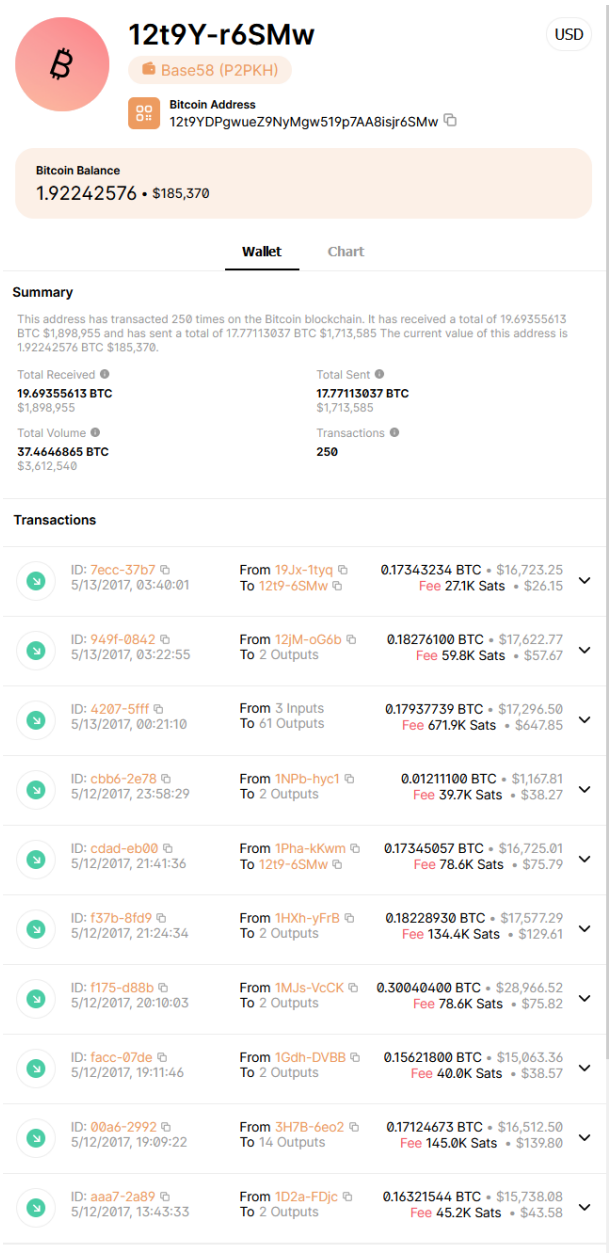


Figure 66: Second hardcoded bitcoin wallet address (Blockchain.com, n.d.)

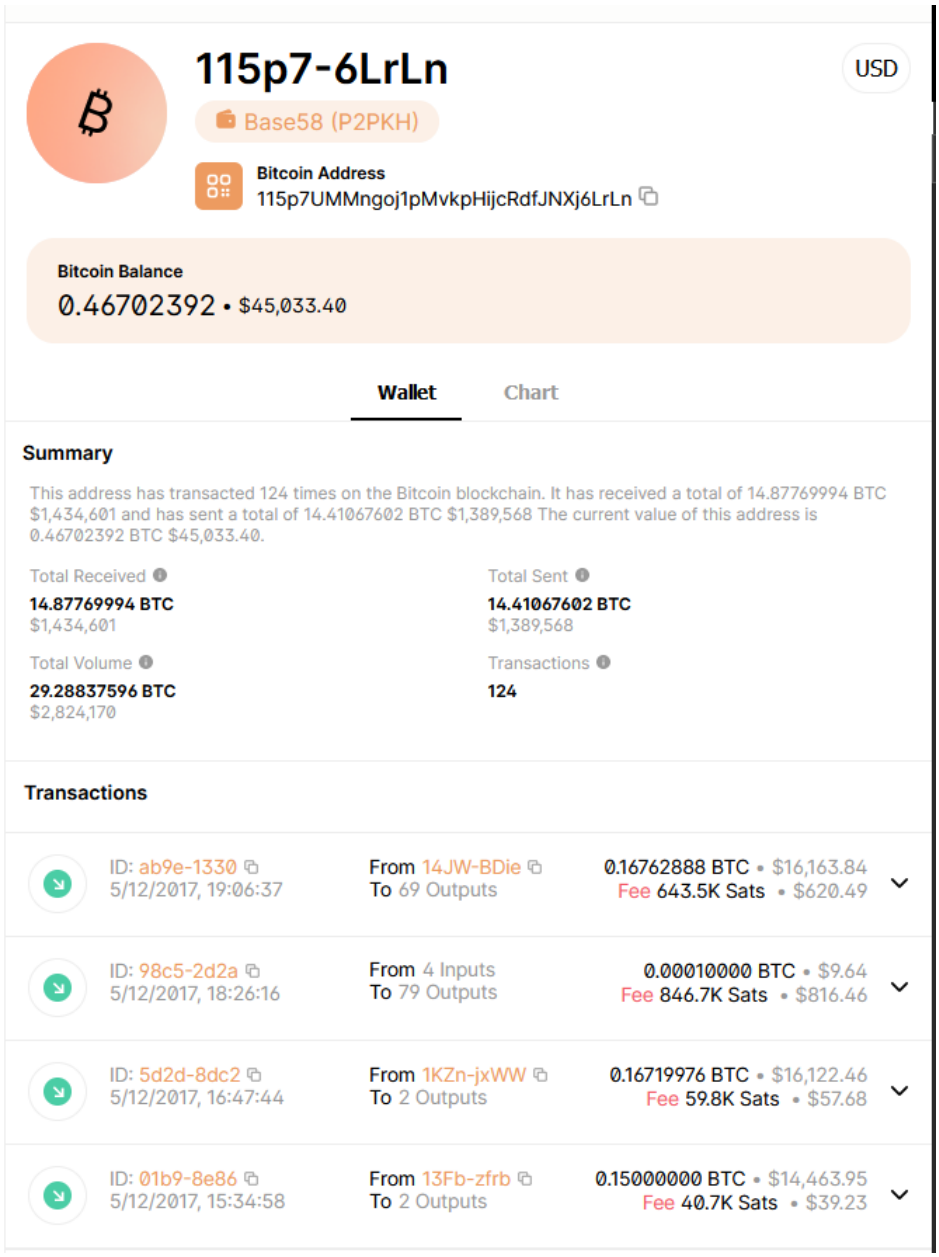


Figure 67: Third hardcoded bitcoin wallet address (Blockchain.com, n.d.)

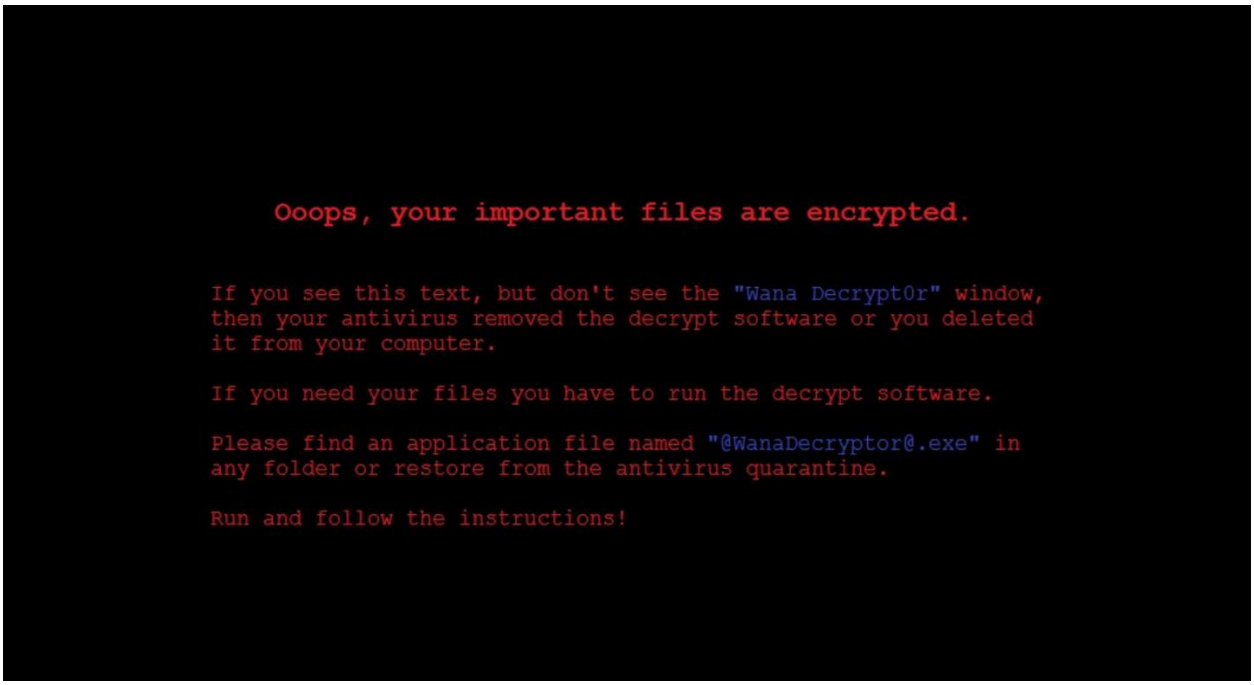


Figure 68: WannaCry wallpaper (desktop icons hidden)